Received: Added at production | Revised: Added at production | Accepted: Added at production

DOI: xxx/xxxx

RESEARCH ARTICLE

Structure-Aware Row Scaling for Block-Structured Mixed-Integer Programs

Mathias Rodríguez Castro

[1]Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

Correspondence
Mathias Rodríguez Castro, Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay.
Email: mathiasr@fing.edu.uy (ORCID: 0009-0002-7235-7677)

Abstract

Generated mixed-integer programs (MIPs) are usually exported to solvers as flat coefficient matrices, yet the generator that builds them knows which constraints are local to physical components, which variables those components own, and which rows couple components through system-wide balances. We ask whether this pre-export metadata can drive positive row scaling without changing the program. We propose a structure-aware row-scaling framework for block-structured MIPs: complete constraints, including right-hand sides, are multiplied by strictly positive factors, so the feasible set, objective, and integer restrictions are preserved exactly. The contribution is not row scaling itself, but using generator metadata to order it—local rows first, then representative block scales, then coupling rows relative to the blocks they connect—which an idealized conditioning analysis shows separates local from coupling imbalance. We evaluate the method on a hydrothermal dispatch platform emitting modular MIPs, treated as a generated block-structured testbed rather than a hydro-specific application; the only metadata required are row roles, block ownership, and coupling incidence. Across three instance classes, two solvers, and two optimality tolerances, the evidence supports three scoped claims: positive row scaling is algebraically exact; structure-aware scaling robustly improves solver-facing numerical diagnostics; and it lowers runtime under Gurobi, with no robust CPLEX gain. A role-blind control locates that runtime effect in per-row local scaling rather than in the row-role metadata: applying the same per-row kernel to every row matches the structure-aware rule on the simpler class and outperforms it on the coupled one. The metadata's demonstrated value is thus in the numerical representation, while the runtime accelerator is a cheap normalization it is not needed to build—a clean dissociation between the rule that best conditions the matrix and the rule that solves fastest. It is best read as a structure-aware numerical preprocessing layer, not a solver-independent acceleration theorem.

KEYWORDS
Numerical scaling, mixed-integer programming, generated formulations, block-structured optimization, engineering optimization, hydrothermal dispatch

PRACTITIONER POINTS

- Model generators know each row's role, block ownership, and coupling incidence—metadata a flat export discards but that can drive numerical preprocessing.
- Positive, structure-keyed row factors applied in a local–block–coupling order improve the solver-facing representation without changing the MIP.
- The runtime benefit is solver-dependent—present under Gurobi, absent under CPLEX—so scaling gains should be validated per solver, not assumed.

1 | INTRODUCTION

A model generator and a solver see different representations of the same optimization problem. The generator builds the model from components: hydro plants, thermal units, market rules, resource limits, demand balances, and discrete operating decisions. While doing so, it knows which rows are local to one component, which variables that component owns, and which constraints link several components. The solver receives the exported version: rows, columns, coefficients, bounds, and tolerances. In that exported matrix, much of the generator's structural knowledge has disappeared.

This loss matters because numerical representation affects computation. Two formulations can describe the same feasible set and objective, yet lead a solver through different presolve reductions, relaxation bases, cuts, incumbent searches, branching decisions, and feasibility checks. Generic scaling routines can inspect coefficient magnitudes after export. They usually cannot tell whether a row is a local turbine restriction, a thermal limit, or a system-wide balance. The generator can make that distinction before export. Mainstream modeling environments—Pyomo, JuMP, GAMS, AMPL—are exactly such generators: they assemble the model component by component but, in their standard export paths, hand the solver a flat matrix that no longer marks which rows are local, which variables a component owns, or which rows couple components. Block-structured extensions of these ecosystems do transport such structure—graph-based and block-decomposition modeling layers, and solver-side decomposition files—but they feed it to decomposition algorithms rather than to numerical preprocessing, which is the gap addressed here.

This is not made obsolete by solver-internal scaling. Commercial solvers already contain sophisticated numerical preprocessing, and the experiments below leave it enabled; the question here is complementary—whether an external layer can supply structural numerical information before the solver sees the matrix as an anonymous list of rows and columns. A positive answer would not replace solver scaling, but change the representation on which the solver's pipeline starts.

The resulting question is simple: can generator-level block information guide positive row scalings that improve the solver-facing representation without changing the MIP? The question is not whether positive row scaling is valid. Multiplying a complete constraint by a strictly positive scalar preserves the feasible set. The question is whether the scaling factors should depend on information that the generator still has: local rows, component blocks, and coupling rows. In this sense, the paper treats pre-export metadata as numerical information rather than only as modeling bookkeeping.

We answer this question with a structure-aware row-scaling framework. The method scales only whole constraints, including their right-hand sides. Hence it preserves the original mixed-integer optimization problem. Its novelty lies in the order and source of the scales, not in claiming that row scaling is new. The generator first scales local component rows. It then summarizes the scale of each locally scaled block. Finally, it scales global coupling rows with the scales of the blocks they connect. Throughout the paper, this local–block–coupling order is the main object of study.

The paper contributes four elements: an algebraically exact positive row-scaling layer for generated block-structured MIPs; a structure-aware rule that uses row roles, component ownership, and coupling incidence; idealized conditioning arguments that motivate the local–block–coupling order; and an empirical evaluation that separates numerical representation from solver-dependent performance. Figure 1 summarizes the contribution at a glance.

The numerical rationale is also local–block–coupling. In an ideal block-diagonal matrix, scalar block scaling can reduce scale separation between blocks but cannot repair a block's intrinsic conditioning; once coupling rows are added, their relevant magnitude is their size relative to the scaled local blocks, not their raw coefficients. Idealized conditioning results make this precise and justify the rule's architecture, without claiming to predict mixed-integer runtime.

We test the method on short-term hydrothermal dispatch, used not as a new formulation but as a demanding testbed: it creates local hydro, thermal, market, shortage, and demand modules linked by global balances, with coefficients mixing water volumes, turbine flows, power outputs, costs, and penalties—block structure with strong scale heterogeneity. The intended generality, however, is the generated block-structured MIP, not the hydrothermal equations. The layer requires only metadata that many modular generators already have—which rows are local, which block owns a variable or constraint, which global rows couple blocks—while reservoir dynamics, turbine curves, and market costs determine the instances but never enter the scaling rule. The hydrothermal model thus supplies a realistic stress test for a layer that targets any generated MIP with local components and sparse global coupling.

The hydrothermal formulation used in the experiments descends from models for the Río Negro hydroelectric complex and extensions to hydrothermal dispatch with Salto Grande, developed by Risso and coauthors, together with a modular C++ implementation used as the experimental platform [1,2,3]. We take the formulation as given and insert the proposed preprocessing layer after model generation and before solver invocation.

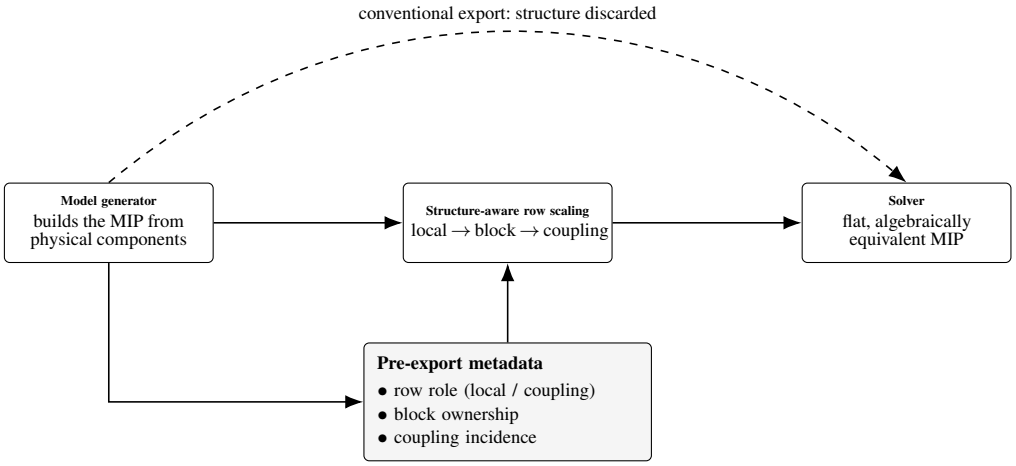


FIGURE 1 The proposed layer in one picture. A generator assembles the MIP from physical components and, in doing so, knows each row's role (local or coupling), which block owns each variable, and how coupling rows link blocks. A conventional export (dashed) hands the solver a flat matrix in which this structure is lost. The structure-aware layer instead reuses that pre-export metadata to choose positive row factors in a local–block–coupling order, then exports an *algebraically equivalent* MIP whose representation is better scaled. The solver still receives a single flat matrix; only its numbers change.

The paper makes three claims of different strength: the algebraic claim is exact (positive row scaling preserves the MIP); the numerical claim is empirical (structure-aware scaling improves solver-facing conditioning in the tested instances); and the performance claim is conditional (under Gurobi positive row scaling often reduces robust runtime or deterministic effort, while under CPLEX 22.1.2.0 it does not). A controlled ablation further separates the two: the runtime lever is a cheap per-row normalization that does not need the row-role metadata, whereas the metadata's measured value is in the conditioning diagnostics. The method improves representation; acceleration depends on how the solver uses it, and the rule that best conditions the matrix is not the rule that solves fastest.

The rest of the paper follows this chain of evidence. Section 2 identifies the research niche. Section 3 defines the generated model structure and testbed. Section 4 gives the scaling method. Section 5 explains how the experiments test the claims. Section 6 reports numerical and runtime evidence. Section 7 interprets that evidence and answers likely objections. Section 8 states the main limitations and reproducibility implications, and section 9 summarizes the contribution.

2 | RELATED WORK

The literature sets the stage but leaves a specific niche. Hydrothermal-dispatch models provide realistic generated block-structured MIPs with heterogeneous units and global balances; numerical-scaling work explains why coefficient magnitudes matter, but its standard tools operate on the exported matrix; and formulation-quality practice shows that algebraic representation can matter as much as the underlying set. The missing argument is narrower than "scaling is useful": when a generator still knows each row's role and each variable's block, that metadata can be used as numerical information before export. The resulting transformation must then be evaluated separately as an algebraic equivalence, a representation change, and a solver-dependent performance intervention.

2.1 | Hydrothermal dispatch and MIP formulations

Hydrothermal dispatch has been studied with dynamic programming, stochastic dual dynamic programming, nonlinear programming, and mathematical programming. Classical references emphasize the trade-off between thermal generation costs,

hydro opportunity costs, demand satisfaction, and intertemporal reservoir management[4,5]. In short-term settings, MIP formulations are attractive because they incorporate discrete decisions, piecewise-linear approximations, local operating constraints, and system-wide balance equations in a single model[6].

This literature motivates the experimental domain but not the novelty claim. The dispatch model is used because it is a representative generated block-structured MIP: local physical modules are created separately, their variables have heterogeneous units, and a smaller set of global rows links the modules through system-wide balances. These are precisely the structural features exploited by the preprocessing layer.

2.2 | Dispatch models used in this study

The computational experiments use a reference formulation from a research line on MIP-based dispatch models for the Uruguayan power system. Risso et al. introduced a mixed combinatorial optimization model for the Río Negro hydroelectric complex[1] and extended the modeling line to hydrothermal dispatch for the Uruguay River hydroelectric plant, with emphasis on Salto Grande and coupled hydrothermal operation[2], and an enriched formulation of the Río Negro model, the closest published reference for the dispatch features used by the Full class[7]. The implementation used here builds on a modular C++ version of the Río Negro dispatch model[3].

This provenance fixes the scope of the experiments. The dispatch formulation, data pipeline, and physical components are taken as given. The proposed method acts after the model has been generated and before the solver is called. Consequently, any observed difference between variants is attributed to an algebraically equivalent representation of the same generated MIP, not to a modified dispatch model.

2.3 | Numerical scaling and solver-facing representation

Scaling is a standard preprocessing operation in linear and mixed-integer optimization. Multiplying a constraint by a positive factor preserves the feasible region but changes the numerical magnitudes passed to the solver. These magnitudes can affect feasibility tolerances, relaxation stability, presolve transformations, cut generation, basis quality, and the numerical behavior of LP relaxations. Practical guidelines for difficult LP and MIP models warn that large coefficient ranges and poor scaling can harm solver performance even when the mathematical formulation is correct[8,9].

This paper uses that observation in a restricted way: it does not claim coefficient compression is itself an objective, nor that a smaller coefficient range must speed up branch-and-bound. A MIP solver is a pipeline—presolve, internal scaling, root relaxation, cuts, heuristics, incumbents, branching, node processing, termination—whose stages can respond differently to the same external scaling. The experiments therefore distinguish exported coefficient-range ratios from solver-level conditioning diagnostics and from runtime, a distinction central to interpreting the Gurobi–CPLEX contrast.

2.4 | Generic equilibration and its limitation for generated models

Generic equilibration procedures balance rows and columns according to matrix norms. This is a mature subfield of numerical linear algebra. Geometric-mean and arithmetic-mean (ℓ_∞) scaling normalize each row or column by an average of its coefficient magnitudes[10]; Curtis–Reid scaling chooses logarithmic diagonal factors that minimize a least-squares measure of coefficient spread[11]; Sinkhorn–Knopp iteration drives a nonnegative matrix toward a doubly stochastic form by alternating row and column normalization[12]; and Ruiz-type scaling iteratively equilibrates row and column norms through diagonal transformations[13]. These methods are the standard tools and are already embedded in solver presolve and internal scaling[14]. Their computational payoff, however, is known to be irregular: the largest empirical study of LP scaling to date found that no scaling method consistently improves—and can even degrade—simplex performance across solvers[15], an early warning that coefficient compression alone is not the lever. Such methods are useful because they are model-agnostic and can be applied after export. That strength is also their limitation in the setting considered here. A flat-matrix method can see coefficient magnitudes and sparsity, but it is not told whether a row is a local turbine restriction, a thermal bound, a market-cost relation, a shortage constraint, or a system-wide balance.

Against this background, the proposed rule is not a new equilibration kernel: its local row factor is, in isolation, a geometric mean of the per-row coefficient extremes, as in geometric-mean scaling. What is new is *where the scales come from and in what order*—the factors are keyed to generator metadata (row role, block ownership, coupling incidence) rather than the flat matrix, and applied in a local–block–coupling order so coupling rows are normalized relative to the already-scaled blocks they connect, not their raw coefficients. A flat method (Ruiz, Curtis–Reid, geometric-mean, Sinkhorn–Knopp) cannot make this local-versus-coupling distinction, because role information is exactly what export discards. Concretely, take two rows with identical coefficients: a flat method cannot tell them apart and scales them identically, whereas the structure-aware rule need not. If one is a local block row and the other a coupling row, the local row is scaled by its own coefficients while the coupling row is scaled relative to the *already-scaled* blocks it links—so a coupling row with small raw coefficients can still receive a large factor when it joins a poorly scaled block, and conversely. Identical numbers, different roles, different factors: that is the lever a flat matrix cannot expose. The model stays monolithic, as with generic scaling: an external, auditable layer that uses generator metadata before the exported matrix loses it.

2.5 | Formulation quality, generated structure, and decomposition

The MIP literature also stresses that formulation quality matters: two algebraically related formulations can describe the same modeling intention but lead to different relaxations, numerics, and search behavior[6]. The present work is adjacent to that tradition but changes a different object. It does not strengthen the formulation, add valid inequalities, alter variables, or change the feasible set. It changes only the positive row multipliers applied to complete constraints.

The method is also adjacent to structural decomposition. Like Dantzig–Wolfe, Benders, or Lagrangian approaches, it recognizes local blocks and global linking rows. Unlike those methods, it does not decompose the optimization problem, introduce a master problem, or exchange prices or cuts between subproblems. The solver still receives one monolithic MIP. The use of block structure is numerical rather than algorithmic decomposition: block metadata guide the representation presented to the solver.

2.6 | Remaining niche

The resulting contribution is deliberately narrow: it tests whether local rows, block scales, and coupling rows should be treated as different numerical objects before a generated block-structured MIP is exported. The performance comparison rests on distributional evidence over heterogeneous instances rather than a single runtime average, using performance profiles[16], paired tests, deterministic-effort metrics, and solver-level conditioning diagnostics.

3 | MATERIALS AND METHODS: GENERATED MODULAR FORMULATION AND TESTBED

We now define the objects that the scaling rule changes and the objects it must leave unchanged. The generator supplies the block structure; the preprocessing layer changes the row magnitudes; the solver then receives an algebraically equivalent MIP. The dispatch model itself is not new. What matters here is the structure that the generator exposes and the original scale in which all feasibility and objective checks are evaluated after the solve. We consider generated modular MIP instances formulated as

$$\begin{aligned} \min_{x} \quad & c^{\top} x \\ \text{s.t.} \quad & Ax \le b, \\ & x_j \in \mathbb{Z}, \qquad j \in \mathcal{I}, \\ & x_j \in \mathbb{R}, \qquad j \notin \mathcal{I}. \end{aligned} \tag{1}$$

In the general generated setting, the vector x collects all variables created by the modules, while A and b contain both local component constraints and global linking constraints. In the hydrothermal testbed used here, x includes generation, reservoir, flow, spillage, shortage, and operating variables. Equalities are represented either explicitly or by pairs of inequalities, depending on the solver interface.

The notation is intentionally application-agnostic: the layer uses the generator's structural map—row role, block ownership, coupling incidence—not hydrological, thermal, or economic semantics. The hydrothermal variables above explain the physical origin of the coefficients; they are not assumptions of the scaling method.

3.1 | Hydrothermal dispatch testbed and scope

The computational testbed uses the reference model generated by the modular dispatch platform developed around Río Negro and Salto Grande hydrothermal dispatch models[1,2,3]. The application is useful because it exhibits the target structure: local modules with heterogeneous physical units and global rows that couple those modules through system-wide balances. The hydraulic, thermal, and market-cost constraints are those of the reference model and its implementation.

A caveat about those heterogeneous units is in order, because it is tempting to read the imbalance as artificial. Unit heterogeneity (reservoir volumes in hm^3, flows in m^3/s, power in MW) lives on the *columns*, and rescaling units is, mathematically, column scaling: an exact reformulation for continuous variables that cannot touch integer columns at all ($x \in \mathbb{Z}$ with $x = s\,x'$ breaks integrality), so in a MIP unit normalization is necessarily partial. Our column-normalizing baseline (Ruiz+Cols) is in fact the weakest variant in runtime, and the base is no ill-scaled straw man: it already runs under the solver's own internal equilibration (section 6.7). The lever that moves runtime in this testbed is therefore on the rows, not the units; the row-only scoping decision is revisited as a limitation in section 8.

The method acts on the generated algebraic model, not the physical dispatch formulation: every transformation must preserve the feasible set and let objective values, constraint violations, and operational variables be read in the original scale after solution. It never uses a hydro-specific rule (reservoir priority, turbine efficiency, unit commitment) to choose factors—those details generate the rows, and only their structural role in the block-coupled MIP is used afterward.

3.2 | Block structure

The method targets models generated by components or agents. Each component $i \in \mathcal{B}$ creates local variables x_i, contributes local constraints $A_i x_i \leq b_i$, and appears in a smaller number of global constraints. Let $\mathcal{R}_i$ denote the local row index set of component i. In the hydrothermal testbed, these components include hydro, thermal, market, and demand modules. If u_{loc} stacks all local variables and g denotes an aggregate coupling variable, the row structure can be written schematically as

$$\begin{bmatrix} D & 0 \\ 0 & I_T \\ L & -I_T \end{bmatrix} \begin{bmatrix} u_{\text{loc}} \\ g \end{bmatrix} \sim \begin{bmatrix} b \\ d \\ 0 \end{bmatrix}, \qquad D = \operatorname{diag}(A_1, \ldots, A_B). \tag{2}$$

The first block contains local component restrictions. The second block represents an aggregate system restriction. The third block contains coupling equations that link local component decisions with the aggregate system variable. In the dispatch application, these rows correspond to demand and balance relationships. More generally, absorbing the aggregate variable and its defining rows into the coupling block, the coefficient matrix takes the block form

$$A = \begin{bmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_B \\ L_1 & L_2 & \cdots & L_B \end{bmatrix}, \qquad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_B \\ h \end{bmatrix}. \tag{3}$$

The matrices A_i encode local restrictions. The matrices L_i encode each block's contribution to the global coupling rows, such as demand balance in the hydrothermal testbed.

This structure suggests three scaling decisions: scale local rows inside each block; compute representative block scales after local row scaling; and scale global coupling rows with those block scales. The goal is to reduce harmful scale heterogeneity without erasing the modular meaning of the formulation.

3.3 | Algebraic equivalence and spectral diagnostics

We restrict the preprocessing layer to positive row scalings. This restriction makes the transformation auditable: variables keep their original units, and the generated MIP remains the same optimization problem. Let $D_r = \operatorname{diag}(d_1, \ldots, d_m)$, with $d_r > 0$ for every row. We replace (A, b) by

$$\widetilde{A} = D_r A, \qquad \widetilde{b} = D_r b. \tag{4}$$

For every vector x,

$$Ax \leq b \quad \Longleftrightarrow \quad D_r A x \leq D_r b, \tag{5}$$

because each component of the inequality is multiplied by a strictly positive scalar. Thus positive row scaling preserves the feasible set, the integer restrictions, and the objective function. Any difference observed after preprocessing must therefore come from the numerical representation seen by the solver, not from a different mathematical MIP.

The theoretical arguments below use the spectral condition number as a diagnostic of linear-algebraic sensitivity. For a real matrix M, define

$$\kappa_2(M) = \frac{\sigma_{\max}(M)}{\sigma^{+}_{\min}(M)}, \tag{6}$$

where $\sigma_{\max}(M)$ is the largest singular value and $\sigma^{+}_{\min}(M)$ is the smallest positive singular value. For rectangular matrices, this ratio is used only as a diagnostic over the nonzero singular spectrum; in the square benchmarks below the minimum is taken over the full spectrum, so that singularity is reported as an infinite condition number. This diagnostic does not characterize the full difficulty of a MIP. A MIP solver also depends on presolve, cuts, branching, primal heuristics, incumbent discovery, feasibility tolerances, and integrality. The role of κ_2 is narrower: it provides an idealized matrix argument explaining why local rows and coupling rows should not be scaled in exactly the same way.

3.4 | Ideal block scaling

The block structure is not only a modeling convenience. It identifies what scalar scaling can and cannot repair. In a purely block-diagonal matrix $D = \operatorname{diag}(A_1, \ldots, A_B)$, a single scalar per block can remove differences of scale between blocks, but it cannot improve the intrinsic conditioning of an individual block. The following benchmark makes this statement precise in the ideal square nonsingular case.

Proposition 1 (Ideal scalar scaling of block-diagonal matrices)**.** *Let $D = \operatorname{diag}(A_1, \ldots, A_B)$, where every A_i is square and nonsingular. Let $M_i = \sigma_{\max}(A_i)$ and $m_i = \sigma_{\min}(A_i) > 0$. Then*

$$\min_{\alpha_i > 0} \kappa_2(\operatorname{diag}(\alpha_1 I, \ldots, \alpha_B I) D) = \max_i \kappa_2(A_i). \tag{7}$$

One minimizer is

$$\alpha_i = \frac{1}{\sqrt{M_i m_i}} = \frac{1}{\sqrt{\sigma_{\max}(A_i)\sigma_{\min}(A_i)}}. \tag{8}$$

Proof. For fixed positive scalars α_i, the scaled matrix is block diagonal:

$$\operatorname{diag}(\alpha_1 I, \ldots, \alpha_B I) D = \operatorname{diag}(\alpha_1 A_1, \ldots, \alpha_B A_B).$$

The singular values of a block-diagonal matrix are the union of the singular values of its blocks. Therefore, for every i,

$$\kappa_2(\operatorname{diag}(\alpha_1 A_1, \ldots, \alpha_B A_B)) \geq \kappa_2(\alpha_i A_i) = \kappa_2(A_i).$$

Taking the maximum over i gives the lower bound

$$\kappa_2(\operatorname{diag}(\alpha_1 A_1, \ldots, \alpha_B A_B)) \geq \max_i \kappa_2(A_i)$$

for every admissible block scaling.

Now choose $\alpha_i = (M_i m_i)^{-1/2}$. Then the largest and smallest singular values of $\alpha_i A_i$ are

$$\sigma_{\max}(\alpha_i A_i) = \sqrt{\frac{M_i}{m_i}} = \sqrt{\kappa_2(A_i)},$$
$$\sigma_{\min}(\alpha_i A_i) = \sqrt{\frac{m_i}{M_i}} = \frac{1}{\sqrt{\kappa_2(A_i)}}.$$

Because the singular values of the complete block-diagonal matrix are obtained by pooling the singular values of its blocks,

$$\sigma_{\max}(\operatorname{diag}(\alpha_1 A_1, \ldots, \alpha_B A_B)) = \sqrt{\max_i \kappa_2(A_i)},$$

and

$$\sigma_{\min}(\operatorname{diag}(\alpha_1 A_1, \ldots, \alpha_B A_B)) = \frac{1}{\sqrt{\max_i \kappa_2(A_i)}}.$$

The resulting condition number is $\max_i \kappa_2(A_i)$, matching the lower bound. □

This proposition is not used as an algorithmic recipe. The generated MIPs considered in this paper contain rectangular local matrices, inequalities, bounds, redundant rows, and integer variables; computing singular-value-based block scalings would not be a practical external preprocessing rule for the exported MIP. The proposition instead justifies the architecture of the method. First, poor scale separation between blocks should be treated differently from poor conditioning inside a block. Second, after local row scaling, each block should contribute a representative scale before global coupling rows are processed. The implemented rules therefore replace the ideal singular-value quantities (M_i, m_i) with coefficient-based proxies that are cheap to compute from the generated model. These propositions are design principles, not performance theorems. They should not be read as predictive guarantees for the implemented MIP preprocessing rule; they motivate the distinction between local rows, block scales, and coupling rows.

3.5 | Effect of global coupling rows

Local block scaling does not exhaust the numerical issue because the generated model is not purely block diagonal. In the hydrothermal dispatch testbed, global rows couple local agent decisions through demand and balance equations. The relevant schematic block is

$$A_{\text{bal}} = \begin{bmatrix} D & 0 \\ L & -I_T \end{bmatrix}, \tag{9}$$

where $D = \operatorname{diag}(A_1, \ldots, A_B)$ represents the local blocks and L collects the coefficients by which local decisions enter the global balance. If D is nonsingular, then

$$A_{\text{bal}} = \begin{bmatrix} I & 0 \\ LD^{-1} & -I_T \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & I_T \end{bmatrix}. \tag{10}$$

This factorization separates local scale from coupling scale. The matrix L by itself is not the relevant object: the coupling felt by the balance row after the local block scales are taken into account is LD^{-1}. Thus a row with moderate entries in L can still be numerically influential if some local block contains weak directions.

The next result makes the dependence on the effective coupling norm explicit.

Proposition 2 (Conditioning of a triangular coupling factor)**.** *Let $R \in \mathbb{R}^{m \times n}$ and define*

$$T_R = \begin{bmatrix} I_n & 0 \\ R & I_m \end{bmatrix}. \tag{11}$$

If $\rho = \|R\|_2$, then

$$\kappa_2(T_R) = \left(\frac{\sqrt{\rho^2 + 4} + \rho}{2} \right)^2. \tag{12}$$

The sign of the identity block is immaterial: since $\left[\begin{smallmatrix} I & 0 \\ R & -I \end{smallmatrix}\right] = \operatorname{diag}(I, -I)\left[\begin{smallmatrix} I & 0 \\ -R & I \end{smallmatrix}\right]$ and orthogonal factors preserve singular values, the same formula applies to the factor with $-I_T$ in equation (10)*.*

Proof. Let $R = U\Sigma V^\top$ be a singular value decomposition of R, with U and V orthogonal. Orthogonal multiplication does not change singular values, and hence T_R has the same singular values as

$$\begin{bmatrix} V^\top & 0 \\ 0 & U^\top \end{bmatrix} \begin{bmatrix} I_n & 0 \\ R & I_m \end{bmatrix} \begin{bmatrix} V & 0 \\ 0 & U \end{bmatrix} = \begin{bmatrix} I_n & 0 \\ \Sigma & I_m \end{bmatrix}.$$

For each singular value σ of R, the nontrivial part reduces to the two-dimensional block

$$B_\sigma = \begin{bmatrix} 1 & 0 \\ \sigma & 1 \end{bmatrix}.$$

The squared singular values of B_σ are the eigenvalues of

$$B_\sigma^\top B_\sigma = \begin{bmatrix} 1+\sigma^2 & \sigma \\ \sigma & 1 \end{bmatrix},$$

namely

$$\lambda_\pm(\sigma) = \frac{\sigma^2 + 2 \pm \sigma\sqrt{\sigma^2+4}}{2}.$$

Their product is one, and $\lambda_+(\sigma)$ is increasing for $\sigma \geq 0$. Additional directions associated with zero singular values of R contribute singular values equal to one. Therefore the largest squared singular value of T_R is $\lambda_+(\rho)$ and the smallest squared singular value is $1/\lambda_+(\rho)$. Consequently,

$$\begin{aligned} \kappa_2(T_R) &= \lambda_+(\rho) \\ &= \frac{\rho^2 + 2 + \rho\sqrt{\rho^2+4}}{2} \\ &= \left(\frac{\sqrt{\rho^2+4} + \rho}{2} \right)^2. \end{aligned}$$

□

Applied to (10), the proposition shows that the balance row should not be treated as an ordinary local row. Its numerical effect depends on the effective coupling proxy $\left\| LD^{-1} \right\|_2$, not only on the raw magnitude of the entries in L. Moreover, if a common positive factor γ is applied to the coupling row, the scaled block satisfies

$$\begin{bmatrix} D & 0 \\ \gamma L & -\gamma I_T \end{bmatrix} = \begin{bmatrix} I & 0 \\ \gamma L D^{-1} & -I_T \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & \gamma I_T \end{bmatrix}. \tag{13}$$

Thus γ changes both the effective coupling term and the relative scale of the aggregate variable in the balance equation: pushing γ down improves the triangular factor of equation (13) but degrades its diagonal factor, and vice versa. This is the trade-off behind the proposed coupling treatment—global rows are scaled after local rows, using block-level representative scales, rather than by an isolated row norm—and the augmented rule resolves it explicitly by minimizing the product of the two factors (equation (21) in section 4.2). In the implemented variants, D^{-1} and singular values are replaced by coefficient-based block proxies, yielding a cheap operational approximation to the ideal coupling measure.

3.6 | From ideal diagnostics to implementable proxies

The two propositions establish *which* numerical distinctions a structure-aware rule must preserve, and thereby justify the architecture. They imply three design requirements: local scale imbalance should be reduced before global coupling rows are diagnosed; blocks should be summarized only after their local rows are on a comparable scale; and a coupling row should be scaled relative to the blocks it connects, not only relative to its own raw coefficient norm. The implemented method realizes exactly these distinctions through coefficient-based proxies: they are cheap to compute from the generated model before export, auditable because they depend only on visible row and block metadata, and they avoid expensive or unstable singular-value information from the rectangular, redundant, inequality-based matrices that arise in generated MIPs. The row factor based on

the geometric mean of the smallest and largest nonzero magnitudes is a robust coefficient-scale proxy that centers each eligible row without changing variables, bounds, integrality, or the objective.

The coupling rule implements the same logic. In proposition 2, the relevant ideal object is the effective coupling magnitude after local scaling, a term of the form LD^{-1}; since generated blocks are rectangular, possibly redundant, and not inverted by the preprocessing layer, the block scale s_i used below is a representative coefficient scale for block i computed after local row scaling, and the coupling proxy divides each coupling coefficient by the representative scale of the block owning its column. This is a cheap, auditable surrogate for asking whether the coupling row is large relative to the local numerical scale of the blocks it links.

We state the scope of this argument once and plainly: the propositions describe square, nonsingular blocks under exact SVD scaling, the proxies act on rectangular, inequality-based matrices, and we establish no bound relating the proxy scale to the ideal spectral factor in proposition 1. The propositions therefore motivate the design, and its empirical validity is verified directly in the controlled synthetic suite of section 5.2, where blocks are small, singular values are cheap to compute, and conditioning is measurable by construction.

This bridge also explains why the proposed method is row-based. Positive row scaling preserves the original units and domains of all variables; integer and binary variables do not require reinterpretation, and objective coefficients are left in their original economic units. Column scaling can be useful as a generic numerical device, so a Ruiz-plus-column baseline is included in the experiments. It is not part of the proposed structure-aware layer because the central claim is that row-role and coupling metadata alone already provide useful pre-export numerical information while keeping the transformation algebraically transparent.

4 | NUMERICAL SCALING METHOD

The scaling method keeps the same structural roles visible throughout: local rows, blocks, and coupling rows. Local rows receive row factors first. Blocks then inherit representative scales from their scaled local rows. Coupling rows receive their factors last, using the blocks they connect. This order is the algorithmic expression of the numerical argument in sections 3.4 and 3.5 and of the proxy interpretation in section 3.6.

This section defines the numerical transformations tested in the results. The base variant is the unscaled model generated by the platform. The two proposed variants use the same local–block–coupling architecture but differ in whether the right-hand side participates in the scale diagnosis. Ruiz-type variants provide generic flat-matrix baselines. This organization makes two contrasts explicit. The comparison between SA-Aug and SA-Mat isolates the role of right-hand-side information within the same architecture; the comparison with Ruiz-type baselines tests whether generator-level row roles add information beyond generic flat-matrix equilibration.

4.1 | Base formulation

The Base variant is passed to the solver without external scaling. Internal solver presolve and internal scaling remain enabled unless stated otherwise. All speedups, objective deviations, and equivalence checks are computed relative to this variant on the same instance, solver, and gap setting.

4.2 | Augmented structure-aware scaling

The first proposed variant, denoted SA-Aug, computes row factors from both coefficient magnitudes and right-hand sides. The motivation is that a generated row may have moderate coefficients but a right-hand side in a very different physical or economic scale. Such a row can still lead to a poorly balanced numerical representation.

For each local row r, define

$$\mathcal{S}_r^{A,b} = \left\{ |a_{rj}| : a_{rj} \neq 0 \right\} \cup \left\{ |b_r| : b_r \neq 0 \right\}. \tag{14}$$

If $\mathcal{S}_r^{A,b} \neq \emptyset$, let

$$M_r^{A,b} = \max \mathcal{S}_r^{A,b}, \qquad m_r^{A,b} = \min \mathcal{S}_r^{A,b}. \tag{15}$$

The local row factor is

$$\alpha_r^{A,b} = \frac{1}{\sqrt{M_r^{A,b} m_r^{A,b}}}. \tag{16}$$

The scaled row is

$$\alpha_r^{A,b} a_r^\top x \le \alpha_r^{A,b} b_r. \tag{17}$$

The augmented rule is not only a row-by-row normalization. It is a three-stage structure-aware procedure that first rescales local rows, then summarizes the resulting local block scales, and finally uses those block scales to treat coupling rows separately. After local scaling, let $\widetilde{a}_{rj} = \alpha_r^{A,b} a_{rj}$ for local rows. For each local block i, define

$$M_i^{A,b} = \max_{r \in \mathcal{R}_i,\, j:\, \widetilde{a}_{rj} \neq 0} |\widetilde{a}_{rj}|, \qquad m_i^{A,b} = \min_{r \in \mathcal{R}_i,\, j:\, \widetilde{a}_{rj} \neq 0} |\widetilde{a}_{rj}|. \tag{18}$$

The representative scale of block i is

$$s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}. \tag{19}$$

For a coupling row r, let $\beta(j)$ denote the local block that owns column j; columns owned by no block enter at the mean block scale, and components that contribute no local rows enter at unit scale. The effective coupling proxy of row r is

$$\widehat{\rho}_r^{A,b} = \sqrt{\sum_j \left(\frac{|a_{rj}|}{s_{\beta(j)}^{A,b}} \right)^2}, \tag{20}$$

the magnitude of the row relative to the representative scales of the blocks it connects, and $\rho = \max_r \widehat{\rho}_r^{A,b}$ is the worst such magnitude over the coupling rows. SA-Aug then scales the coupling block by a *single* factor $\gamma^{A,b}$, applied to every coupling row and chosen to minimize the conditioning surrogate that the idealized analysis suggests:

$$\gamma^{A,b} = \arg\min_{\gamma} \Phi(\gamma), \qquad \Phi(\gamma) = \kappa_T(\gamma\rho)\, \kappa_B(\gamma), \qquad \kappa_B(\gamma) = \frac{\max(\bar{M}, \gamma)}{\min(\bar{m}, \gamma)}, \tag{21}$$

where $\kappa_T(\cdot)$ is exactly the bound of equation (12) on the triangular coupling factor, κ_B is the diagonal spread once the aggregate variable enters at scale γ, and $[\bar{m}, \bar{M}]$ is the spread of the block scales after local normalization. The minimizer is found on a logarithmic grid $\gamma \in [10^{-6}, 10^{6}]$, and the stage is skipped when the effective coupling is already moderate (ρ below a fixed threshold) or when the minimizer falls inside the near-identity band of the safeguards. The grid search resolves explicitly the trade-off exposed by equation (13): decreasing γ improves the triangular factor but degrades the diagonal one, and Φ is the product that the factorization bounds. The released implementation evaluates these proxies from coefficient summaries of the generated model; the exact evaluation order is documented in the reproducibility repository. This sequence is the main structural feature of the proposed method: local physical scales are used first, block scales are estimated after local normalization, and global coupling rows are scaled only after the blocks they connect have been put on comparable numerical footing.

4.3 | Matrix-only structure-aware scaling

The second proposed variant, denoted SA-Mat, uses the same row–block–coupling architecture but excludes b from the diagnosis. The right-hand side is still multiplied by the selected row factor to preserve equivalence, but it does not influence factor selection.

For each local row r, define the nonzero coefficient extremes

$$M_r^A = \max_{j:\, a_{rj} \neq 0} |a_{rj}|, \qquad m_r^A = \min_{j:\, a_{rj} \neq 0} |a_{rj}|. \tag{22}$$

If the row has at least one nonzero coefficient, the row factor is

$$\alpha_r^A = \frac{1}{\sqrt{M_r^A m_r^A}}. \tag{23}$$

The scaled row is

$$\alpha_r^A a_r^\top x \le \alpha_r^A b_r. \tag{24}$$

This factor centers the smallest and largest nonzero coefficient magnitudes of the row around a common scale. Unlike Euclidean-norm scaling, it does not directly depend on the number of nonzeros in the row.

Let $\widetilde{a}_{rj} = \alpha_r^A a_{rj}$ after local row scaling. For each local block i, define

$$M_i^A = \max_{r\in\mathcal{R}_i, j:\,\widetilde{a}_{rj}\neq 0} |\widetilde{a}_{rj}|\,, \qquad m_i^A = \min_{r\in\mathcal{R}_i, j:\,\widetilde{a}_{rj}\neq 0} |\widetilde{a}_{rj}|\,. \tag{25}$$

The representative block scale is

$$s_i^A = \sqrt{M_i^A m_i^A}. \tag{26}$$

No additional multiplier is applied to all rows or columns of block i at this stage. The quantity s_i^A is an auxiliary representative scale: it is used only to normalize the contribution of block i when the coupling-row proxy is computed.

For a coupling row r, with the same ownership map $\beta(\cdot)$ and the same convention for blockless columns as in section 4.2, define

$$\widehat{\rho}_r^A = \sqrt{\sum_j \left(\frac{|a_{rj}|}{s_{\beta(j)}^A}\right)^2}. \tag{27}$$

If $\widehat{\rho}_r^A > 0$, the coupling-row factor is per row,

$$\gamma_r^A = \frac{1}{\widehat{\rho}_r^A}, \tag{28}$$

clipped to $[10^{-8}, 10^8]$ and subject to the safeguards below. When the block extremes of equation (25) are computed, the implementation restricts them to coefficients inside a fixed moderation window $(10^{-6}, 10^6)$, so a single extreme coefficient cannot distort s_i^A.

The comparison between SA-Aug and SA-Mat isolates the empirical value of including the right-hand side in the scaling diagnosis. Both variants preserve the same structural architecture; they differ only in whether b participates in factor selection.

4.4 | Safeguards

All applied scaling factors—the local row factors and the coupling-row factors in the augmented or matrix-only variants—are subject to two per-factor safeguards, applied independently to each factor. The representative block scales s_i are auxiliary quantities used to compute coupling-row proxies; they are not applied as independent row or column scalings.

4.4.0.1 | *Near-identity threshold*

A factor is applied only if it departs from one by at least a factor of $u = 2$; factors falling in the open band $(1/u, u) = (0.5, 2)$ are treated as negligible and skipped:

$$\theta \text{ is applied} \iff \left(\theta \le \frac{1}{u}\right) \text{ or } (\theta \ge u)\,, \qquad u = 2. \tag{29}$$

This prevents re-scaling rows or coupling rows that are already at a reasonable scale.

4.4.0.2 | *Admissible coefficient range*

A factor is applied only if it keeps every affected coefficient within the window $[\varepsilon_{\min}, \varepsilon_{\max}] = [10^{-9}, 10^9]$. Writing $c_{\min}$ and $c_{\max}$ for the smallest and largest magnitudes of the affected coefficients, the factor θ is admitted only if

$$\begin{aligned} \varepsilon_{\min} \le c_{\min}\,\theta, \qquad & c_{\max}\,\theta \le \varepsilon_{\max}, \\ [\varepsilon_{\min}, \varepsilon_{\max}] = [10^{-9},\ 10^{9}]. & \end{aligned} \tag{30}$$

Otherwise, the factor is not applied and the corresponding row or coupling row is left at its base scale. The coupling factor is additionally hard-clipped to $\gamma \in [10^{-8},\ 10^8]$ prior to this check.

Because each admitted factor is a strictly positive multiplier applied to a complete constraint—both its coefficients and its right-hand side—the feasible set and the objective value are preserved exactly. The safeguards therefore restrict only the numerical aggressiveness of the transformation, never its equivalence to the base model.

Algorithm 1 Augmented structure-aware row scaling

Require: Matrix A, right-hand side b, row roles (local/coupling), block ownership $\beta(\cdot)$, coupling incidence
Ensure: Scaled matrix $\widetilde{A}$, scaled right-hand side $\widetilde{b}$

Initialize row factors $d_r \leftarrow 1$
for each local block $i \in \mathcal{B}$ **do**
  for each local row $r \in \mathcal{R}_i$ **do**
    Build $\mathcal{S}_r^{A,b}$ from nonzero coefficients and nonzero right-hand side
    if $\mathcal{S}_r^{A,b} \neq \emptyset$ **then**
      Compute $M_r^{A,b}$ and $m_r^{A,b}$ using equation (15)
      Set $\alpha_r^{A,b} \leftarrow 1/\sqrt{M_r^{A,b} m_r^{A,b}}$
      if the safeguards admit $\alpha_r^{A,b}$ **then**
        Update $d_r \leftarrow d_r \alpha_r^{A,b}$
      end if
    end if
  end for
end for
Compute the locally scaled coefficients $\widetilde{a}_{rj} = d_r a_{rj}$
Compute block scales $s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}$ using equation (19)
Compute $\widehat{\rho}_r^{A,b}$ for every coupling row using equation (20); set $\rho \leftarrow \max_r \widehat{\rho}_r^{A,b}$
if ρ exceeds the coupling threshold **then**
  Set $\gamma^{A,b} \leftarrow \arg\min_\gamma \Phi(\gamma)$ over the logarithmic grid $\gamma \in [10^{-6}, 10^{6}]$ (equation (21))
  if $\gamma^{A,b}$ lies outside the near-identity band **then**
    Update $d_r \leftarrow d_r \gamma^{A,b}$ for every coupling row r
  end if
end if
Return $\widetilde{A} = \mathrm{diag}(d)A$ and $\widetilde{b} = \mathrm{diag}(d)b$

4.5 | Ruiz-type baselines

We compare the proposed variants with two generic baselines. The first, Ruiz, applies iterative row equilibration based on the maximum norm. The second, Ruiz+Cols, combines Ruiz-type row equilibration with continuous-column scaling and physical normalization. These baselines separate the effect of generic coefficient compression from the effect of exploiting model structure.

Ruiz is deliberately not a weak foil: max-norm equilibration is a standard, strong flat-matrix scheme, and Ruiz+Cols additionally grants it the column-scaling lever the structure-aware rule forgoes. The baselines isolate a narrow question—whether row roles and coupling incidence carry numerical information that a flat-matrix equilibrator, however well tuned, cannot recover once export has discarded them; the separate "could the solver have done this itself" control is the internal-scaling baseline of section 6.7. Every variant hands the solver the same monolithic MIP; only the pre-export row factors differ.

The final implementation uses

$$K = 4 \tag{31}$$

Ruiz iterations. Binary and integer variables are not rescaled unless explicitly stated, so that bounds, objective interpretation, and integrality remain unchanged.

4.6 | Algorithmic summary

The matrix-only algorithm follows the same local and block stages with the coefficient-only set $\{|a_{rj}| : a_{rj} \neq 0\}$ and the representative block scales and coupling proxies of equations (26) and (27); its coupling stage differs in granularity, assigning each coupling row its own factor $\gamma_r^A = 1/\widehat{\rho}_r^A$ (equation (28)) instead of the single grid-selected $\gamma^{A,b}$. In the present manuscript,

TABLE 1 Three instance classes test the method on progressively different generated hydrothermal models.

Class	Configuration	Main role	Instances
Simple	Río Negro + fixed demand + fixed dispatch	Basic numerical behavior	204
Full	Río Negro nonlinear + Salto Grande + thermal units + market	Largest integrated model	34
SG-Ter-Mer	Salto Grande + thermal units + market	Coupled hydrothermal subset	68

The counts report the maximum operational instance pools with available real input data. Scenario tables may show different n values because each aggregate row uses the valid runs available for that solver–gap–variant combination. Pairwise speedups and tests use matched base–variant pairs only.

TABLE 2 Four solver–tolerance scenarios separate solver dependence from gap-tolerance effects.

Scenario	Solver	Relative MIPGap
1	Gurobi	10^{-2}
2	Gurobi	10^{-3}
3	CPLEX	10^{-2}
4	CPLEX	10^{-3}

Deterministic time is reported when available: `Work` units for Gurobi and ticks for CPLEX.

SA-Mat is interpreted as an ablation of the right-hand-side information: it preserves the same local-row and coupling-row architecture, but the right-hand side does not enter the factor-selection rule.

5 | EXPERIMENTAL DESIGN

5.1 | Instance classes and scenarios

The experiments are organized around four reader questions: does scaling preserve the original optimization problem, does it improve the numerical representation, does that change solver behavior, and does the effect survive changes in class, solver, and tolerance? The study uses three instance classes generated by the same modular hydrothermal platform, built from operational instances with available real input data. The pools are balanced by construction: every variant is attempted on every instance of a class (204 Simple, 68 SG-Ter-Mer, 34 Full), so no instance is dropped for one variant and kept for another. Valid-run counts still differ slightly across class–scenario–variant cells because individual runs can fail with an error status (an aborted solve, a missing log, an unavailable conditioning read)—a small fraction, about 1–3% per class in the Gurobi scenarios, recorded per run in the artifacts. Descriptive metrics use all valid runs for a row (so a cell may report, say, 195–199 of 204 Simple instances), whereas speedups, tests, and all base comparisons use matched base–variant pairs in which both runs succeeded, and are therefore unaffected by the unbalanced descriptive counts.

Each measurement is tied to the part of the argument it can support: algebraic equivalence is checked by descaling and original-scale feasibility and integrality tests; the numerical claim by exported coefficient-range summaries and solver-level conditioning diagnostics; the performance claim by matched base–variant speedups, deterministic effort, performance profiles, and primal–dual progress; and solver dependence by separate Gurobi and CPLEX scenarios at two tolerances (the same distinctions return as claim–evidence–limitation in table 20).

Table 1 separates the three pools: Simple gives the largest sample for distributional evidence, Full is the smallest and most integrated model, and SG-Ter-Mer isolates a coupled hydrothermal subset—so the averages and paired tests do not come from a single homogeneous population.

The class names are application-specific, but the contrast is structural: whether the rule behaves consistently as the generated MIP moves from a simpler modular model to larger or more strongly coupled ones. The conclusions are thus stated at two levels—empirical evidence from hydrothermal dispatch, methodological object the use of generator metadata in block-structured engineering MIPs.

We evaluate four solver–gap scenarios. The time limit is one hour per solve in every case. Internal solver presolve and internal scaling are left enabled, because the goal is to test whether external preprocessing helps in a realistic solver pipeline. All main experiments were run on ClusterUY, a collaborative high-performance computing platform for scientific computing in Uruguay [17].

TABLE 3 The computational environment fixes solver versions, time limits, and default presolve settings for all comparisons.

Item	Configuration
Cluster	ClusterUY, partition `ute`
Processor	Intel Xeon Gold 6138
Threads	16
Memory limit	12 GB
Time limit	1 hour per solve
Gurobi version	13.0.1 [18]
CPLEX version	22.1.2.0 [19]
Presolve	solver default, enabled

Solver-internal scaling and presolve were left at their default enabled settings unless otherwise stated. CPLEX 22.1.2.0 is the version available in the ClusterUY environment used for the long runs; conclusions involving CPLEX should therefore be interpreted as solver-version and environment specific.

The four scenarios in table 2 vary two dimensions separately: the stopping tolerance and the solver pipeline. Scenarios 1–2 test Gurobi at two MIPGap values; scenarios 3–4 repeat the comparison under CPLEX and serve as a solver contrast. The configuration in table 3 matters because the proposed preprocessing is tested on top of each solver's default numerical pipeline. The CPLEX conclusions below are therefore statements about this version and ClusterUY environment, not claims about all CPLEX releases or parameter choices.

5.2 | Controlled synthetic block-structured benchmark

The hydrothermal testbed evaluates the proposed preprocessing rule on realistic operational generated MIPs. To complement this real-data setting, we define a controlled synthetic benchmark whose purpose is diagnostic rather than physical. The synthetic instances are not intended to model a power system. They isolate the numerical mechanism targeted by the method by allowing local-row scale imbalance and coupling-row scale imbalance to be varied independently.

Each synthetic instance consists of B local blocks. Block i contains continuous activity variables $y_i \in \mathbb{R}^{n_y}_+$ and binary activation variables $z_i \in \{0,1\}^{n_z}$. A nonnegative slack vector s is added to the global coupling constraints to guarantee feasibility. The generic synthetic MIP is

$$\begin{aligned}
\min_{y,z,s} \quad & \sum_{i=1}^{B} c_i^\top y_i + \sum_{i=1}^{B} q_i^\top z_i + P\mathbf{1}^\top s \\
\text{s.t.} \quad & R_i y_i + G_i z_i \le r_i, \qquad i = 1,\dots,B, \\
& 0 \le y_{ik} \le U_{ik} z_{ik}, \qquad i = 1,\dots,B, \quad k = 1,\dots,n_y, \\
& \sum_{i=1}^{B} L_i y_i + \sum_{i=1}^{B} H_i z_i + s \ge d, \\
& s \ge 0, \qquad z_i \in \{0,1\}^{n_z}.
\end{aligned} \tag{32}$$

The first group of constraints defines local block restrictions, the second group links continuous activity variables to binary activation variables, and the third group creates global coupling rows. The penalty P is chosen large enough that the slack variable is used only when the activated block capacity cannot satisfy the synthetic demand.

The unscaled coefficients are generated from bounded random distributions with controlled sparsity. Local matrices R_i and G_i define block-internal resource constraints. Coupling matrices L_i and H_i define the contribution of each block to the global rows. The generator records row metadata

$$\texttt{row\_type} \in \{\texttt{local}, \texttt{coupling}\}, \qquad \texttt{block\_id} \in \{1,\dots,B\}, \tag{33}$$

and column metadata identifying the block associated with each variable. This metadata is the same information required by the structure-aware scaling rules.

TABLE 4 Controlled synthetic benchmark factors isolate the numerical mechanisms targeted by the scaling rule.

Factor	Levels used	Controlled object	Purpose
Scale pattern	none, local, coupling, mixed	Row class affected by artificial scaling	Separate local imbalance from coupling imbalance
Severity S	0, 3, 6	Range of row multipliers 10^{-S}–10^{S}	Test mild and severe numerical heterogeneity
Blocks and size	$B = 10$, $n_y = n_z = 8$	Number and size of generated local components	Keep a fixed block structure while varying row-scale imbalance
Rows and seeds	8 local rows per block, 3 coupling rows, 20 seeds	Random coefficients and sparsity patterns	Estimate distributional behavior rather than one instance

The synthetic instances are not physical power-system models. They are controlled block-structured MIPs designed to test whether the preprocessing rule responds to the type of scale imbalance it is designed to exploit. The reported synthetic grid uses the middle size $B = 10$, the four patterns, the three severity levels, and twenty random seeds per pattern–severity cell.

Scale imbalance is introduced after the clean synthetic MIP has been generated. For each eligible row r, a positive row multiplier

$$\theta_r = 10^{\xi_r}, \qquad \xi_r \sim \mathcal{U}[-S, S], \tag{34}$$

is drawn and applied to the entire row, including its right-hand side. Therefore, the scaled representation is algebraically equivalent to the clean synthetic MIP. The severity parameter S controls the possible scale range. We use

$$S \in \{0, 3, 6\}, \tag{35}$$

corresponding to no artificial imbalance, moderate imbalance, and strong imbalance.

Four scale patterns are considered:

1. **None:** no artificial row scaling is applied.
2. **Local:** only local block rows are multiplied by factors (34).
3. **Coupling:** only global coupling rows are multiplied by factors (34).
4. **Mixed:** both local block rows and global coupling rows are multiplied.

The "none" pattern is a negative control: a useful preprocessing rule should not create large artificial changes when the representation is already reasonably scaled. The "local" and "coupling" patterns isolate the two numerical objects distinguished by the proposed method. The "mixed" pattern approximates the combined heterogeneity observed in generated models. These controlled patterns are mechanism checks. They verify that the implementation reacts correctly when the source of imbalance is known by construction, but they are not an operational local-only versus coupling-only ablation on the real hydrothermal instances.

The reported synthetic runs follow the factor grid summarized in table 4, using the five scaling variants in section 5.3, the same original-scale feasibility and objective checks, Gurobi with relative MIPGap 10^{-3}, and a 600 second time limit. They are intentionally small: the benchmark is a controlled mechanism check for numerical conditioning and algebraic equivalence, not a performance experiment. Runtime claims are reserved for the operational hydrothermal instances.

5.3 | Scaling variants

The compared variants are: Base, the original formulation; SA-Aug, the augmented structure-aware scaling rule; SA-Mat, the matrix-only structure-aware rule with safeguards; Ruiz, generic Ruiz-type row equilibration with $K = 4$ iterations; and Ruiz+Cols, Ruiz-type row scaling plus continuous-column scaling and physical normalization. Binary and integer variables are not rescaled.

5.4 | Additional controls: internal scaling, sensitivity, and stage ablation

Three further controls sharpen the attribution of the main comparison. They reuse the same matched-pairs design, the same instance classes, and the same original-scale verification, and they leave the five variants above unchanged.

5.4.0.1 | *Solver-internal-scaling baseline*

To separate "structure-aware external scaling helps" from "any extra scaling pass helps, and the solver could have produced it," we run the *unscaled* model under the solver's own scaling controls and compare it, within each scenario, against the structure-aware variants run under default internal scaling. For Gurobi we sweep the scaling flag over its admissible settings (no scaling, equilibration, geometric, and aggressive modes); for CPLEX we sweep its read-scaling parameter (none, default equilibration, aggressive). If a tuned internal setting matches the structure-aware runtime on the same instances, the distinctive claim collapses to generic coefficient compression; if it does not, the gain cannot be attributed to a scaling pass the solver could have produced itself. Internal scaling and presolve remain enabled throughout, as in the main experiments.

5.4.0.2 | *Hyperparameter sensitivity*

The method fixes three constants: the near-identity band $u = 2$ (equation (29)), the admissible coefficient window $[\varepsilon_{\min}, \varepsilon_{\max}] = [10^{-9}, 10^{9}]$ (equation (30)), and the number of Ruiz iterations $K = 4$. We report a sensitivity analysis over $u \in \{1.5, 2, 4\}$, $K \in \{2, 4, 8\}$, and the window over $\{[10^{-6}, 10^{6}], [10^{-9}, 10^{9}], [10^{-12}, 10^{12}]\}$ on the Simple class under Gurobi. The K sweep also tests whether the irregular matched behavior of the Ruiz baseline is an artifact of a particular iteration count.

5.4.0.3 | *Operational stage ablation*

The synthetic benchmark isolates local and coupling imbalance by construction, but the operational comparison evaluates the full local–block–coupling architecture. To attribute the operational effect to its stages, we additionally run two single-stage ablations on the hydrothermal classes—local rows scaled but coupling rows left at base scale, and coupling rows scaled but local rows left at base scale—against the same base. Comparing the full architecture with each ablation identifies how much of the Gurobi effect comes from local row normalization and how much from the coupling-row treatment.

5.5 | Metrics and reporting conventions

The analysis reports three families of metrics. First, feasibility preservation is checked after descaling the solver solution and evaluating it on the original MIP. The primary post-solve check is the maximum violation of the original constraints, together with the maximum integrality violation. A run is labeled feasible in original scale if the maximum row violation is at most 10^{-4} and the maximum integer violation is at most 10^{-5}. Recomputing the objective in original units is retained as an auxiliary consistency diagnostic, but it is not the primary feasibility criterion. Objective agreement with the base is measured by

$$\Delta z_{\text{rel}} = \frac{|z_v - z_{\text{base}}|}{\max\{1, |z_{\text{base}}|\}}. \tag{36}$$

Second, numerical behavior is a primary outcome, not only an explanatory variable for speed. We distinguish three quantities. The first is the spectral condition number used in section 3.3 to motivate the rule in an idealized linear-algebraic setting. The second is the exported coefficient-range ratio

$$\rho_\kappa(v) = \frac{(\max|a|/\min|a|)_v}{(\max|a|/\min|a|)_{\text{base}}}, \tag{37}$$

where the extrema are computed over nonzero coefficients in the generated matrix. This is a static proxy and should not be interpreted as the condition number of the MIP. The third is the solver-level ratio

$$\rho_\kappa^{\text{solver}}(v) = \frac{\kappa_{\text{solver}}(v)}{\kappa_{\text{solver}}(\text{base})}, \tag{38}$$

where κ_{solver} is obtained from the fixed-integer LP after the main MIP solve when available; for CPLEX we also record tree-level KappaStats (maximum sampled basis condition number, unstable and ill-conditioned fractions, numerical attention score). Parsed solver warnings are kept as a descriptive alert, not a ranking criterion. Values below one indicate improvement relative to the base. The paper thus separates the numerical effect from whether it is converted into faster MIP search.

Third, efficiency is measured by wall-clock time, solver time, total time including preprocessing, deterministic time, node counts, iterations, final gap, and the primal–dual integral. For any time measure T_q, the paired speedup is

$$\text{speedup}_q(v) = \frac{T_{q,\text{base}}}{T_{q,v}}, \qquad q \in \{\text{solver}, \text{total}, \det\}. \tag{39}$$

Values above one indicate that variant v is faster than the base. Because solution times are highly skewed, we report shifted geometric means

$$\mathrm{sgm}_s(\{x_i\}) = \exp\left(\frac{1}{n}\sum_i \log(x_i + s)\right) - s, \tag{40}$$

using $s = 1$ s for times and $s = 10$ for node counts. The primal–dual integral is

$$\mathrm{PI} = \frac{1}{T}\int_0^T \gamma(t)\,dt, \tag{41}$$

where $\gamma(t)$ is the relative optimality gap reported by the solver's informational callback, clamped to $[0, 1]$ and set to one before the first incumbent; the integral is accumulated by trapezoids over the callback stream and T is the total solve time. Because callback cadence and gap reporting differ between solvers, the absolute level of PI is comparable within a scenario table but not across solvers. Timeouts are summarized by PAR10: the arithmetic mean of solver time over the instance pool, with each run that reaches the limit counted as ten times the time limit.

5.6 | Statistical analysis

The statistical analysis is paired by construction: every variant is compared with the base formulation on the same instance, solver, and MIPGap tolerance. Since MIP solution times are strongly skewed and include censored observations, solver-time comparisons use the Wilcoxon signed-rank test on matched, non-double-timeout pairs. Pairs in which both the base and the variant reach the time limit are treated as censored for this paired test; their global effect is still reflected in PAR10 and in the Dolan–Moré performance profiles. Alongside the test statistic W, we report the raw p-value, the Benjamini–Hochberg adjusted value q_{BH}, and Cliff's δ as an effect size. The Benjamini–Hochberg correction is applied within each scenario (solver $\times$ tolerance), over the family of all four scaled variants across the three instance classes (twelve paired tests); the tables display the two structure-aware variants, while q_{BH} reflects that full family. With the convention used here, negative values of δ indicate that the variant tends to be faster than the base, whereas positive values indicate deterioration.

To connect numerical and algorithmic effects, we report conditioning ratios by class and variant and compute Spearman correlations between solver speedup and two ratios: the exported coefficient-range ratio ρ_κ and the solver-level ratio $\rho_\kappa^{\mathrm{solver}}$. The question is not only whether either is significant, but whether the solver-level diagnostic is more informative than the static proxy on the same matched observations; we compare the two overlapping dependent correlations with the Williams–Steiger test. To rule out a size artifact, we also report the first-order partial rank correlation between $\rho_\kappa^{\mathrm{solver}}$ and speedup controlling for instance size (loaded nonzeros), pooled across classes and variants (size is nearly constant within a class). The partial coefficient ($\rho_{\mathrm{partial}} \approx -0.25$, $n = 1185$) is essentially unchanged from the unconditional value (≈ -0.26, explaining only $\rho^2 \approx 6\%$ of the speedup variance), so the weak conditioning–speedup association is not a size artifact; the same comparison shows the solver-level diagnostic predicts speedup markedly better than the static proxy (Williams–Steiger $Z \approx 6.1$, $p < 10^{-8}$), while either association stays weak—consistent with conditioning being at most a partial mediator (section 7).

6 | COMPUTATIONAL RESULTS

The results follow the hierarchy of claims. We first check whether the solver-facing representation improves. We then ask whether Gurobi converts that improvement into lower runtime or deterministic effort. Finally, we use CPLEX as a contrast to test whether the performance effect is solver-independent.

The order matters because numerical improvement and algorithmic acceleration are related but not identical outcomes—a conditioning table alone would imply too much, a timing table alone too little—so the numerical and algorithmic evidence must be read together. The main pattern is clear but bounded: under Gurobi, the structure-aware variants improve solver-facing conditioning and usually reduce time or deterministic effort; under CPLEX 22.1.2.0, the same external scalings do not produce a robust runtime gain.

6.1 | Original-scale equivalence across the operational campaigns

Before the per-scenario results, we report the descaling audit promised in section 5.5 for the four operational campaigns. After every completed run, the solver solution is descaled and re-evaluated on the untransformed MIP; table 5 pools the four scaling variants per scenario and instance class and reports the share of runs meeting the primary criterion (maximum row violation at most 10^{-4}, maximum integrality violation at most 10^{-5}), together with the median and worst row violation and the objective deviation Δz_{rel} of equation (36). Three facts hold uniformly over the 4,806 variant runs. The integrality criterion is met in every run (worst violation 9.9×10^{-6}). The objective recomputed on the original model from the descaled solution matches the solver incumbent to at most 1.0×10^{-6}, so the descaling trail itself is exact to solver precision. Every exceedance discussed below is therefore a property of the solution returned by the solver, not an artifact of rescaling or of the bookkeeping that undoes it.

Feasibility pass rates are high wherever the solver returns clean solutions: 89% of runs pooled over the Gurobi campaigns and 94% of the CPLEX runs outside the Simple class meet the strict criterion. The visible exception is Simple under CPLEX (32% pass). The unscaled base is the control that locates the cause: it fails the same checker at essentially the same rate (25–26% pass), on essentially the same instances (151 of the 154 base failures at 1% MIPGap recur under SA-Aug), on the same rows (balance equalities with zero right-hand side), and with the same magnitude (median residual near 3×10^{-4}), while Gurobi passes the identical instances at 99.5%. The exceedance is attributable to the solutions CPLEX returns on this class, not to the external scaling, and is consistent with feasibility tolerances being enforced on the solver's internally scaled and presolved model rather than on the original rows. A fixed absolute threshold also explains the largest worst-case entries: a residual at solver tolerance on a scaled row maps back to ε/α_r on the original row, and the extremes sit on rows of large magnitude. The worst absolute violation in the table (7.9×10^2, Simple under CPLEX at 0.1%) carries a relative residual of at most 1.6×10^{-6} on a constraint whose right-hand side exceeds 5×10^8 in original units, and the SG-Ter-Mer maximum of 1.1×10^{-1} is below 4.5×10^{-10} relative to its row. The remaining Gurobi exceedances are a bounded tail: the scaled variants exceed the absolute threshold in 13–15% of Simple runs (22–28% for the Ruiz row variant, 6–15% for the structure-aware variants), and in the worst cell, Full under SA-Aug at 1% (79% pass), the seven failing runs violate original rows by 2.8×10^{-4} to 2.6×10^{-3} (all but one on equality rows) with zero integrality violation and objective agreement with the base within the gap band.

Objective agreement must be read against the MIPGap: both members of a pair are solved to the campaign gap, so deviations up to roughly twice the gap are alternative near-optima, not model changes, and the median Δz_{rel} is at or below gap level in every cell. Deviations beyond twice the gap affect 72 of the 4,806 runs (1.5%), concentrate on a handful of instances, and split into two patterns, both internally consistent in the sense above. First, Ruiz-family runs occasionally claim optimality at grossly suboptimal incumbents: a single Full instance drives the 1.8×10^2 maxima, with the row-Ruiz run terminating at gap zero on an incumbent whose original-scale cost is 1.9×10^8 against a base incumbent of 1.1×10^6, under both solvers and both gaps. Second, some unscaled-base runs claim optimality at objectives that verified-feasible scaled runs beat by wide margins: on one Simple instance all four variants reach costs 73% below the base's claimed optimum, and on one Full instance under CPLEX all four variants agree within 0.5% on an objective the base misses by a factor of six, the value the Gurobi campaigns confirm. The audit flags failures in both directions; the worst worse-than-base deviation of a structure-aware variant among claimed-optimal runs is 3.6%, on one Full instance. Runs that fail to complete are excluded from the matched pairs throughout; runs that merely exceed the strict original-scale threshold are retained in the performance analyses, since filtering on an outcome correlated with the treatment would bias the runtime comparison.

6.2 | Scenario 1: Gurobi with 1% MIPGap

At the looser Gurobi tolerance, the evidence favors the structure-aware family. SA-Mat gives the lowest shifted geometric solver time in Simple and SG-Ter-Mer, and it improves deterministic effort in all three classes. SA-Aug remains the strongest descriptive rule in Full. The Ruiz-type baselines do not tell the same story: they may change coefficient ranges, but their runtime behavior is irregular.

Table 6 shows the descriptive gains span more than one statistic: SA-Mat leads $\mathrm{sgm}\,T$ and $\mathrm{PAR10}$ in Simple and SG-Ter-Mer and SA-Aug leads Full, with no comparable class-consistent pattern for the Ruiz baselines. The SG-Ter-Mer $\mathrm{PAR10}$ margin, however, rests on a single timeout-rescued instance (see the table notes); for that class the shifted-geometric-mean gain is the robust signal. Figure 2 shows the per-instance speedup distributions, figure 3 contrasts the two conditioning diagnostics against speedup, and figure 4 gives the corresponding Dolan–Moré profiles.

TABLE 5 Original-scale equivalence audit of the four operational campaigns. Each solver solution is descaled and re-evaluated on the untransformed MIP; a run passes if the maximum row violation is at most 10^{-4} and the maximum integrality violation at most 10^{-5} (section 5.5). Row violations are absolute, in original units; Δz_{rel} is equation (36) against the base incumbent on the same instance.

Scenario	Class	n[a]	Pass (%)	Base (%)[b]	viol. med.	viol. max.	Δz_{rel} med.	Δz_{rel} max.
Gurobi, 1%	Simple	792	86.7	99.5	2.9×10^{-6}	4.9×10^{-3}	3.8×10^{-5}	8.7×10^{-1}
	Full	134	89.6	97.1	4.8×10^{-6}	2.6×10^{-3}	9.2×10^{-4}	1.8×10^{2}
	SG-Ter-Mer	270	98.1	100.0	6.8×10^{-7}	1.1×10^{-1}	7.9×10^{-5}	2.1×10^{-2}
Gurobi, 0.1%	Simple	787	85.4	99.5	3.5×10^{-6}	1.2	2.7×10^{-5}	8.7×10^{-1}
	Full	122	90.2	100.0	4.8×10^{-6}	4.5×10^{-3}	6.3×10^{-5}	1.8×10^{2}
	SG-Ter-Mer	255	98.4	100.0	6.6×10^{-7}	1.1×10^{-1}	1.5×10^{-14}	2.1×10^{-2}
CPLEX, 1%	Simple	816	32.0	24.5	2.3×10^{-4}	4.4×10^{-1}	1.4×10^{-15}	7.3×10^{-2}
	Full	136	80.9	75.8	1.4×10^{-5}	1.3×10^{-3}	1.0×10^{-3}	5.3×10^{1}
	SG-Ter-Mer	272	100.0	100.0	6.8×10^{-7}	1.4×10^{-5}	4.3×10^{-14}	9.7×10^{-3}
CPLEX, 0.1%	Simple	816	32.6	26.0	2.4×10^{-4}	7.9×10^{2}	4.3×10^{-15}	1.1×10^{-1}
	Full	134	83.6	84.8	1.3×10^{-5}	6.2×10^{-4}	7.0×10^{-6}	5.3×10^{1}
	SG-Ter-Mer	272	100.0	100.0	6.9×10^{-7}	1.4×10^{-5}	9.1×10^{-15}	2.0×10^{-3}

[a] Completed runs pooled over SA-Aug, SA-Mat, Ruiz, and Ruiz+Cols; runs that failed to complete are excluded, and three runs lacking verification output are counted as non-passing.
[b] Pass rate of the unscaled base under the same checker (n per scenario: 195–204 Simple, 31–34 Full, 62–68 SG-Ter-Mer).

TABLE 6 At 1% MIPGap, Gurobi converts structure-aware scaling into lower robust-efficiency metrics in the Simple and SG-Ter-Mer classes.

Class	Variant	n	sgm T	med. sp.$^{\text{det}}$	PAR10	PI
Simple	Base	197	4.05	1.00	11.6	0.245
	SA-Aug	197	2.20	1.07	2.6	0.242
	SA-Mat	198	**2.03**	1.06	**2.4**	0.241
	Ruiz	198	8.27	0.88	26.7	0.300
	Ruiz+Cols	199	7.52	1.02	27.6	**0.233**
Full	Base	34	82.93	1.00	3,384.8	0.120
	SA-Aug	34	**70.56**	**1.37**	3,307.8	0.075
	SA-Mat	34	77.82	1.15	**2487.2**	**0.055**
	Ruiz	33	126.94	1.13	4,566.5	0.263
	Ruiz+Cols	33	75.89	1.32	4,696.3	0.239
SG-Ter-Mer	Base	67	14.14	1.00	572.7	0.664
	SA-Aug	67	8.39	1.32	562.2	0.566
	SA-Mat	67	**6.75**	**1.40**	**21.0**	**0.531**
	Ruiz	68	14.40	1.00	567.3	0.639
	Ruiz+Cols	68	18.35	0.78	585.1	0.662

Notes. sgm T: shifted geometric mean of solver time in seconds. med. sp.$^{\text{det}}$: median deterministic speedup with respect to the base model. PI: primal–dual integral. Lower is better for sgm T, PAR10, and PI; higher is better for med. sp.$^{\text{det}}$. Bold values mark the best descriptive value within each class and metric. In SG-Ter-Mer the large PAR10 separation of SA-Mat reflects a single near-limit instance it solves within the time budget while Base and SA-Aug time out; the shifted-geometric-mean gain (14.1 → 6.8 s) is the more robust evidence for this class, and the PAR10 rescue is not reproduced under the internal-scaling and stage-ablation campaigns (sections 6.7 and 6.8), where the instance times out under all variants.

6.3 | Scenario 2: Gurobi with 0.1% MIPGap

The stricter Gurobi tolerance tests whether the pattern survives a harder stopping criterion, and it largely does: SA-Mat again gives the best shifted geometric solver time in Simple and SG-Ter-Mer and SA-Aug is stronger in Full, while the Ruiz-type variants remain unstable. Since the matrix-only rule uses no right-hand-side information, much of the useful structural scale information, in this testbed, is already present in the coefficient matrix.

Table 7 confirms it: the Gurobi evidence does not hinge on the looser stopping criterion, and the distributional and diagnostic views at this tolerance preserve the same pattern as their 1% counterparts (Supporting Information, Figures S1–S3).

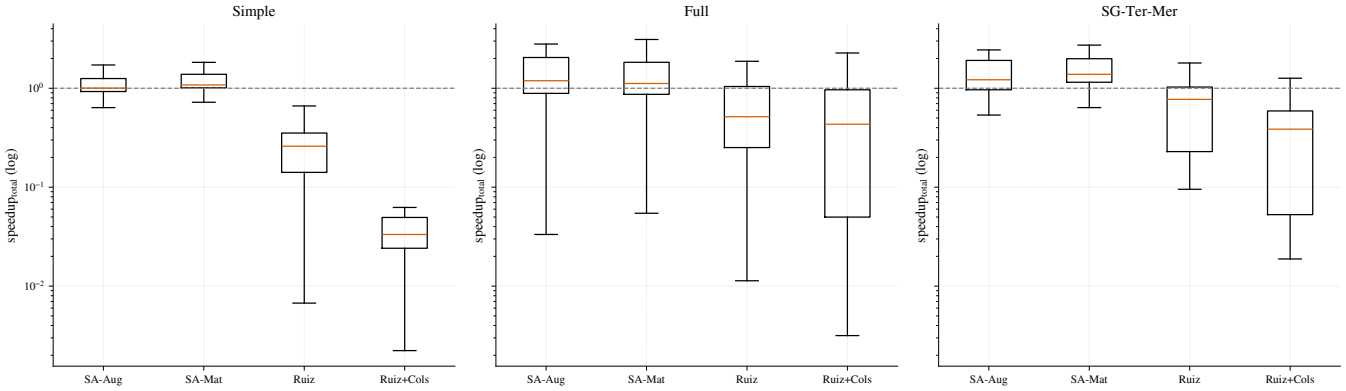
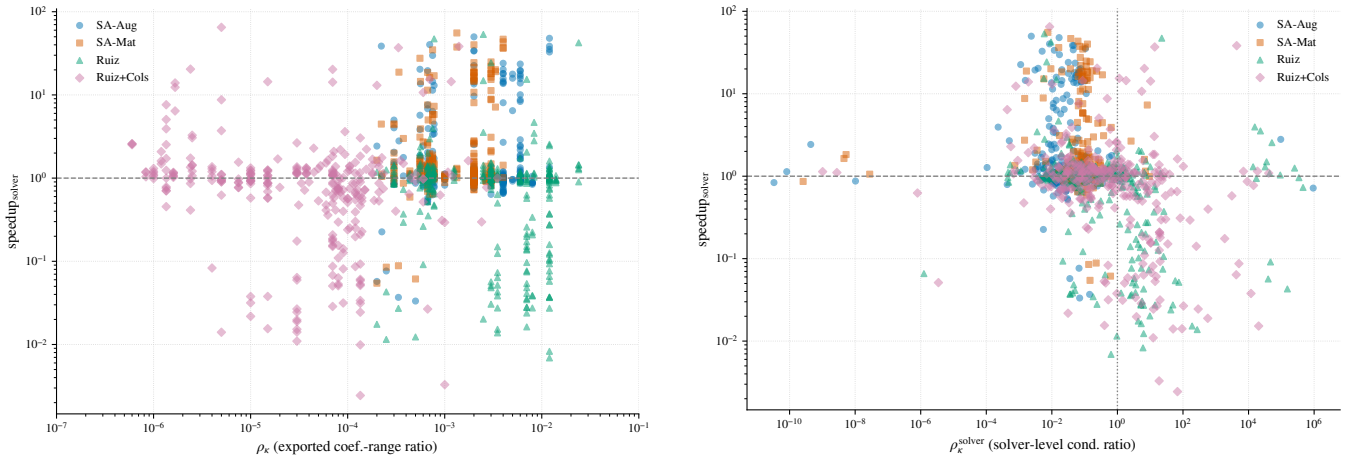


FIGURE 2 At 1% MIPGap, the Gurobi speedup distributions place the structure-aware variants mostly above parity, while Ruiz-type baselines are more dispersed. Values above the parity line indicate acceleration relative to the unscaled model. In all box plots, boxes span the interquartile range and whiskers extend to the most extreme observation within 1.5× IQR; outliers are omitted, since tail behavior is captured by PAR10 and the performance profiles.



(a) Static coefficient-range ratio.

(b) Solver-level conditioning ratio.

FIGURE 3 At 1% MIPGap, static coefficient-range compression and solver-level conditioning tell different stories. The contrast between panels shows why exported matrix compression alone should not be treated as evidence of easier branch-and-bound search. In panel (a) every static ratio lies below one, so the $\rho_\kappa = 1$ reference sits outside the plotted range.

Table 8 separates two numerical notions that are easy to conflate. Both structure-aware variants drive the solver-level ratio well below one (SA-Mat brings $\rho_\kappa^{\text{solver}}$ to 0.07–0.08 in Full and SG-Ter-Mer while compressing ρ_κ much like SA-Aug), whereas Ruiz+Cols may compress the exported matrix while worsening the solver-level diagnostic in Full—so the structured rules are not just generic equilibration in another form.

The paired tests in table 9 support the descriptive Gurobi pattern. The signs are favorable for both structure-aware variants, and after Benjamini–Hochberg adjustment the effect is significant for SA-Mat in Simple and SG-Ter-Mer; in the smaller Full class ($n \leq 34$) the paired evidence for SA-Aug is directionally consistent but does not reach significance ($q_{\text{BH}} = 0.15$), so the Full result is best read as descriptive. The effect sizes are moderate rather than dramatic; the evidence is a distributional shift, not a guarantee that every instance accelerates.

Table 10 adds the aggregate correlation check. The exported coefficient-range ratio is essentially uninformative about speedup, while the solver-level conditioning ratio has the expected negative association in the pooled data—although, decomposed by variant, that association is carried by the Ruiz baselines rather than by the structure-aware rules (see the table notes). The contrast between the two diagnostics is the empirical basis for treating solver-facing diagnostics as more informative

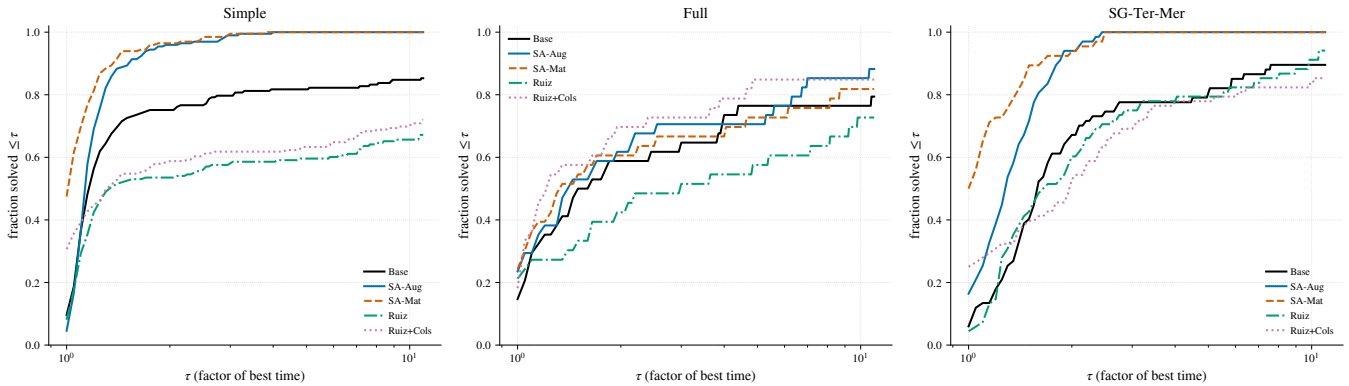


FIGURE 4 At 1% MIPGap, the Dolan–Moré profiles show whether the average Gurobi improvements persist across the instance pool. Curves farther toward the upper-left indicate variants that more often solve close to the best observed time.

TABLE 7 At 0.1% MIPGap, the Gurobi efficiency pattern persists: structure-aware variants remain the strongest structured preprocessing rules.

Class	Variant	n	sgm T	med. sp.$^{\det}$	PAR10	PI
Simple	Base	195	8.28	1.00	28.5	**0.176**
	SA-Aug	195	5.65	**1.11**	28.7	0.180
	SA-Mat	198	**5.44**	1.10	**21.1**	0.182
	Ruiz	199	14.12	0.88	41.5	0.261
	Ruiz+Cols	195	12.55	1.02	48.9	0.199
Full	Base	31	305.30	1.00	5,023.8	0.027
	SA-Aug	30	**226.90**	1.26	**3890.2**	0.028
	SA-Mat	31	306.39	1.12	5,059.3	0.025
	Ruiz	30	341.56	0.87	5,137.5	0.027
	Ruiz+Cols	31	267.10	**1.62**	6,140.1	**0.022**
SG-Ter-Mer	Base	62	14.49	1.00	613.0	0.642
	SA-Aug	63	8.99	1.32	593.7	0.473
	SA-Mat	65	**7.97**	**1.35**	**22.7**	**0.439**
	Ruiz	65	14.27	1.05	586.8	0.633
	Ruiz+Cols	62	19.16	0.78	629.1	0.625

Notes. Bold values mark the best descriptive value within each class and metric; lower is better for sgm T, PAR10, and PI, whereas higher is better for med. sp.$^{\det}$. In SG-Ter-Mer the PAR10 separation of SA-Mat rests on the same single near-limit rescued instance as at the 1% tolerance (see the notes to table 6); the sgm gain is the robust signal for this class.

than exported-matrix summaries; the near-null structure-aware correlations independently reinforce the reading, developed in section 7, that conditioning is at most a partial mediator of their runtime benefit.

Together with tables 8 and 10, these results support a two-step interpretation: structure-aware scaling changes the numerical representation seen by Gurobi, and the solver-level conditioning ratio is much more informative about speedup than the raw coefficient-range ratio.

TABLE 8 Structure-aware scaling improves solver-facing conditioning diagnostics under Gurobi, whereas static coefficient compression alone is insufficient.

Class	Variant	n	median ρ_κ (IQR)	median $\rho_\kappa^{\text{solver}}$ (IQR)	warnings
Simple	SA-Aug	195	0.00 (0.00)	0.04 (0.08)	1.0
	SA-Mat	195	0.00 (0.00)	0.08 (0.08)	1.0
	Ruiz	192	0.01 (0.01)	0.75 (3.94)	2.0
	Ruiz+Cols	190	1.5×10^{-5} (6.7×10^{-5})	1.19 (12.70)	1.0
Full	SA-Aug	30	5.6×10^{-4} (3.7×10^{-4})	0.02 (0.04)	1.0
	SA-Mat	31	5.6×10^{-4} (3.7×10^{-4})	0.07 (0.08)	1.0
	Ruiz	29	5.6×10^{-4} (3.7×10^{-4})	0.47 (2.55)	2.0
	Ruiz+Cols	30	6.0×10^{-4} (8.0×10^{-4})	1.60 (24.82)	0.0
SG-Ter-Mer	SA-Aug	62	6.7×10^{-4} (1.8×10^{-4})	0.01 (0.01)	1.0
	SA-Mat	61	6.7×10^{-4} (1.8×10^{-4})	0.08 (0.02)	1.0
	Ruiz	61	6.7×10^{-4} (1.8×10^{-4})	0.06 (0.06)	1.0
	Ruiz+Cols	59	1.0×10^{-4} (8.9×10^{-5})	0.07 (0.11)	0.0

Notes. Values below one indicate improvement relative to the base on the same scenario. ρ_κ is the exported coefficient-range proxy, whereas $\rho_\kappa^{\text{solver}}$ is computed from the solver-level conditioning diagnostic on the fixed-integer LP. Entries shown as 0.00 are positive ratios below the displayed precision. The warnings column summarizes solver numerical warning flags parsed from logs and is used only as a descriptive diagnostic.

TABLE 9 Paired Gurobi tests show a distributional runtime shift in favor of the structure-aware variants, especially in Simple and SG-Ter-Mer.

Scenario	Class	Variant	W	p	q_{BH}	Cliff's δ
1% MIPGap	Simple	SA-Aug	7535.0	0.0057	0.0097	–0.090
1% MIPGap	Simple	SA-Mat	3011.0	$< 10^{-4}$	$< 10^{-4}$	–0.210
1% MIPGap	Full	SA-Aug	175.0	0.0984	0.1476	–0.082
1% MIPGap	Full	SA-Mat	172.0	0.1406	0.1875	–0.073
1% MIPGap	SG-Ter-Mer	SA-Aug	501.0	0.0001	0.0005	–0.138
1% MIPGap	SG-Ter-Mer	SA-Mat	115.0	$< 10^{-4}$	$< 10^{-4}$	–0.239
0.1% MIPGap	Simple	SA-Aug	6963.0	0.0010	0.0024	–0.077
0.1% MIPGap	Simple	SA-Mat	7069.0	0.0016	0.0028	–0.080
0.1% MIPGap	Full	SA-Aug	97.0	0.0262	0.0393	–0.144
0.1% MIPGap	Full	SA-Mat	173.0	0.7140	0.7140	–0.023
0.1% MIPGap	SG-Ter-Mer	SA-Aug	456.0	0.0004	0.0018	–0.145
0.1% MIPGap	SG-Ter-Mer	SA-Mat	341.0	$< 10^{-4}$	0.0002	–0.158

Notes. Tests are paired against the base formulation on matched instances. Negative Cliff's δ means that the variant tends to be faster. Double-timeout pairs are excluded from the Wilcoxon test and retained through PAR10 and the performance profiles. The effective number of matched pairs equals the full class pool (Simple 195–197, SG-Ter-Mer 61–67) except in Full, where double-timeouts reduce it to 27–32; zero-difference pairs are dropped by the Wilcoxon zero method.

TABLE 10 Solver-level conditioning is more informative about Gurobi speedup than the exported coefficient-range proxy.

Scenario	$\rho_S(\rho_\kappa, \text{sp})$	p	$\rho_S(\rho_\kappa^{\text{solver}}, \text{sp})$	p	Z	p_Z
1% MIPGap	+0.003	0.926	–0.256	3.7×10^{-19}	6.11	$< 10^{-4}$
0.1% MIPGap	+0.006	0.837	–0.267	5.6×10^{-20}	6.60	$< 10^{-4}$

Notes. sp denotes solver speedup. The Williams–Steiger statistic Z compares the two overlapping correlations measured on the same matched observations. The rows pool all four scaled variants; decomposed by variant, the negative solver-level association is carried by the Ruiz baselines ($\rho_S \approx$ –0.33 for Ruiz and –0.36 for Ruiz+Cols at 1%), whereas for the structure-aware variants it is essentially null (SA-Aug –0.06, $p = 0.30$; SA-Mat +0.11, $p = 0.06$), with the same pattern at 0.1%.

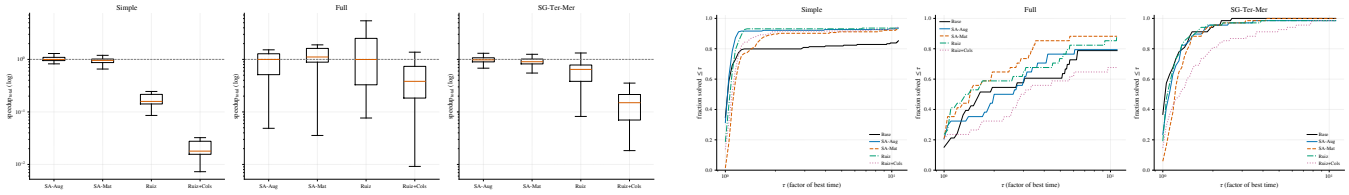
6.4 | Scenario 3: CPLEX with 1% MIPGap

The CPLEX scenarios provide a negative solver contrast to the Gurobi results. The structured variants remain close to parity and do not reproduce the systematic deterministic gains observed under Gurobi. Some descriptive wall-time improvements

TABLE 11 At 1% MIPGap, CPLEX does not reproduce the Gurobi structure-aware acceleration pattern.

Class	Variant	n	sgm T	med. sp.$^{\mathrm{det}}$	PAR10	PI
Simple	Base	204	2.23	1.00	6.2	0
	SA-Aug	204	**1.46**	**1.04**	3.1	0
	SA-Mat	204	1.68	1.03	3.4	0
	Ruiz	204	1.49	0.99	3.6	0
	Ruiz+Cols	204	1.51	0.96	**2.6**	0
Full	Base	33	158.38	1.00	5621.7	3.1×10^{-8}
	SA-Aug	34	165.43	0.98	5546.0	3.2×10^{-8}
	SA-Mat	34	**115.60**	1.00	6497.8	2.3×10^{-8}
	Ruiz	34	115.96	**1.14**	**3506.3**	$\mathbf{2.1 \times 10^{-8}}$
	Ruiz+Cols	34	225.94	0.70	4681.1	5.5×10^{-8}
SG-Ter-Mer	Base	68	**2.21**	1.00	**3.5**	1.0×10^{-10}
	SA-Aug	68	2.37	**1.02**	9.9	1.0×10^{-10}
	SA-Mat	68	2.41	1.00	5.5	1.1×10^{-10}
	Ruiz	68	2.33	1.01	10.3	$\mathbf{9.7 \times 10^{-11}}$
	Ruiz+Cols	68	3.18	0.88	15.2	1.0×10^{-10}

Notes. Bold values mark the best descriptive value within each class and metric. Since the CPLEX statistical tests do not support a robust positive external-scaling effect, these bold entries are descriptive only. PI is accumulated from CPLEX's own callback stream and is comparable within this table but not against the Gurobi tables (section 5.5).



(a) Total-time speedup distribution. (b) Performance profile.

FIGURE 5 At 1% MIPGap, the CPLEX distributional and performance-profile views break the Gurobi pattern. The plots therefore serve as a solver contrast.

appear, especially in Full for SA-Mat, but the paired tests do not support robust structure-aware acceleration. In fact, some CPLEX signs are unfavorable for the structured variants. The correct interpretation is therefore not that CPLEX improves less than Gurobi, but that the present external scaling layer does not reliably improve CPLEX runtime in this environment. Ruiz is sometimes competitive in Full, but its performance is not stable across classes.

For CPLEX, table 11 has to be interpreted differently from the Gurobi tables: the class-consistent structure-aware pattern is absent, and in SG-Ter-Mer the base scale already gives the best shifted geometric time and PAR10. The Simple class shows the descriptive–paired dissociation most clearly: SA-Mat lowers the class sgm (2.23 → 1.68 s) and PAR10 (6.2 → 3.4 s), yet its paired test is significantly *unfavorable* (Cliff's $\delta = +0.125$, table 13): a few hard instances in the tail accelerate enough to move the aggregates while the typical instance slows slightly. Where a CPLEX aggregate improves, it is therefore a tail effect, not a per-instance speedup. Figure 5 shows the corresponding distributional and profile views.

6.5 | Scenario 4: CPLEX with 0.1% MIPGap

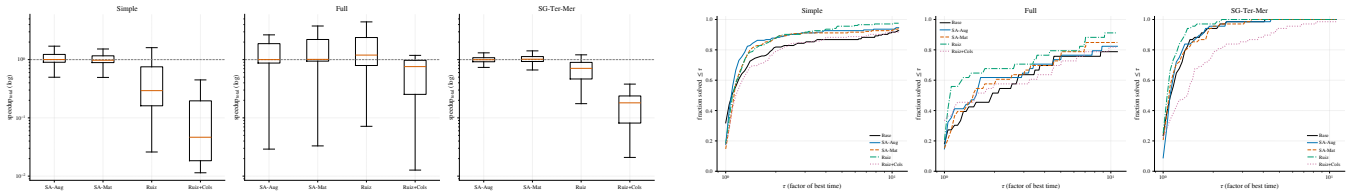
The stricter CPLEX setting confirms the same solver dependence and makes the negative runtime conclusion clearer. The structured variants remain at or near parity in deterministic effort, and the clearest descriptive improvement in shifted geometric solver time comes from Ruiz rather than from the proposed structure-aware rules; the matrix-only rule does lower the shifted geometric time on Full (from 421.6 to 370.5 s), but this is descriptive only, as the paired test on that class is not significant ($p = 0.21$). This is a useful negative result: external scaling may improve some numerical diagnostics without becoming a robust runtime accelerator for CPLEX in this environment.

Table 12 reports the cells and figure 6 the corresponding distributional and profile views.

TABLE 12 At 0.1% MIPGap, CPLEX remains close to parity for the structure-aware variants, confirming solver dependence.

Class	Variant	n	sgm T	med. sp.$^{\mathrm{det}}$	PAR10	PI
Simple	Base	204	5.14	1.00	213.6	1.2×10^{-12}
	SA-Aug	204	4.20	1.02	**188.4**	6.5×10^{-13}
	SA-Mat	204	4.48	1.01	202.5	5.6×10^{-13}
	Ruiz	204	**4.13**	1.00	192.0	$\mathbf{1.0 \times 10^{-15}}$
	Ruiz+Cols	204	5.28	0.94	546.6	7.7×10^{-13}
Full	Base	33	421.59	1.00	9162.0	2.7×10^{-8}
	SA-Aug	34	422.62	1.07	8930.4	2.7×10^{-8}
	SA-Mat	33	370.45	1.05	11186.3	2.3×10^{-8}
	Ruiz	34	**285.10**	**1.28**	**5857.3**	$\mathbf{1.9 \times 10^{-8}}$
	Ruiz+Cols	33	377.41	0.96	1.1×10^{4}	5.2×10^{-8}
SG-Ter-Mer	Base	68	2.82	1.00	18.8	1.1×10^{-10}
	SA-Aug	68	2.84	1.01	20.6	$\mathbf{1.0 \times 10^{-10}}$
	SA-Mat	68	2.84	1.00	15.8	1.1×10^{-10}
	Ruiz	68	**2.61**	**1.02**	**7.5**	1.1×10^{-10}
	Ruiz+Cols	68	4.02	0.85	539.5	1.3×10^{-10}

Notes. Bold values mark the best descriptive value within each class and metric. Under CPLEX, these descriptive minima do not imply a robust positive external-scaling effect.



(a) Total-time speedup distribution.

(b) Performance profile.

FIGURE 6 At 0.1% MIPGap, the CPLEX visual evidence remains mixed under the stricter tolerance, supporting a solver-dependent interpretation.

TABLE 13 Paired CPLEX tests do not support a robust positive runtime effect for the structure-aware variants.

Scenario	Class	Variant	W	p	q_{BH}	Cliff's δ
1% MIPGap	Simple	SA-Aug	9377.0	0.2016	0.3456	–0.058
1% MIPGap	Simple	SA-Mat	7460.0	0.0004	0.0023	+0.125
1% MIPGap	Full	SA-Aug	172.0	0.3357	0.4284	+0.032
1% MIPGap	Full	SA-Mat	131.0	0.1042	0.2083	–0.153
1% MIPGap	SG-Ter-Mer	SA-Aug	1068.0	0.5211	0.5211	–0.003
1% MIPGap	SG-Ter-Mer	SA-Mat	740.0	0.0082	0.0326	+0.055
0.1% MIPGap	Simple	SA-Aug	8765.0	0.0453	0.1213	–0.024
0.1% MIPGap	Simple	SA-Mat	10056.0	0.7230	0.7230	+0.005
0.1% MIPGap	Full	SA-Aug	173.0	0.7140	0.7230	–0.001
0.1% MIPGap	Full	SA-Mat	125.0	0.2079	0.3563	–0.038
0.1% MIPGap	SG-Ter-Mer	SA-Aug	1085.0	0.5908	0.7089	–0.021
0.1% MIPGap	SG-Ter-Mer	SA-Mat	985.0	0.2507	0.3760	–0.011

Notes. Tests are paired against the base formulation on matched instances. Under CPLEX, the effect signs are small, inconsistent, or unfavorable. They do not support a robust positive structure-aware runtime effect. The effective number of matched pairs equals the full class pool (Simple 203–204, SG-Ter-Mer 68) except in Full, where double-timeouts reduce it to 26–29; zero-difference pairs are dropped by the Wilcoxon zero method.

The paired CPLEX tests in table 13 keep the descriptive tables in perspective. Unlike the Gurobi evidence, the signs are small, mixed, and sometimes unfavorable; after multiplicity adjustment they do not support a robust positive runtime effect for the structure-aware variants.

The CPLEX runs nevertheless carry their own conditioning evidence, recorded as tree-level `KappaStats` (section 5.5), and it moves in the same direction as the Gurobi diagnostics. The median sampled-basis condition ratio of the structure-aware

TABLE 14 The best descriptive variant depends on the solver pipeline, not only on the external scaling rule.

Scenario	Simple	Full	SG-Ter-Mer	Interpretation
Gurobi, 1%	SA-Mat	SA-Aug	SA-Mat	The two structure-aware rules dominate descriptively; SA-Mat is strongest in Simple and SG-Ter-Mer, while SA-Aug is strongest in Full.
Gurobi, 0.1%	SA-Mat	SA-Aug	SA-Mat	The stricter gap preserves the same pattern and strengthens the evidence that a matrix-only structural rule can be competitive under Gurobi.
CPLEX, 1%	SA-Aug	SA-Mat	Base	Descriptive winners vary by class; paired tests do not support a robust positive structure-aware effect and some signs are unfavorable.
CPLEX, 0.1%	Ruiz	Ruiz	Ruiz	Ruiz is descriptively strong in shifted geometric time, especially in Full, but this is a CPLEX-pipeline-specific pattern rather than evidence for universal Ruiz dominance.

Notes. The table summarizes the lowest shifted geometric solver time within each class. It is descriptive only; the statistical evidence is reported separately in tables 9 and 13.

TABLE 15 Controlled synthetic benchmark: median condition numbers before scaling and after each variant, with worst original-scale violations across the selected cells.

Pattern	S	κ^{pre} (Base)	κ^{post} (SA-Aug)	κ^{post} (SA-Mat)	κ^{post} (Ruiz)	κ^{post} (Ruiz+Cols)	v_{row}	v_{int}
none	0	1.34×10^{3}	1.09×10^{3}	2.49×10^{3}	1.88×10^{3}	1.88×10^{3}	4.55×10^{-13}	0
local	3	1.84×10^{8}	1.07×10^{3}	4.68×10^{3}	3.15×10^{3}	3.15×10^{3}	2.91×10^{-11}	0
local	6	1.49×10^{14}	1.07×10^{3}	4.02×10^{3}	3.67×10^{3}	3.67×10^{3}	1.49×10^{-8}	0
coupling	3	6.86×10^{5}	1.22×10^{3}	2.49×10^{3}	2.11×10^{3}	2.11×10^{3}	1.75×10^{-10}	0
coupling	6	4.87×10^{8}	1.22×10^{3}	2.49×10^{3}	2.67×10^{3}	2.67×10^{3}	1.12×10^{-7}	2.03×10^{-6}
mixed	3	2.06×10^{8}	1.18×10^{3}	4.68×10^{3}	3.24×10^{3}	3.24×10^{3}	1.16×10^{-10}	0
mixed	6	1.52×10^{14}	1.22×10^{3}	4.02×10^{3}	4.58×10^{3}	4.58×10^{3}	1.31×10^{-6}	1.28×10^{-7}

Notes. Values are medians by pattern, severity, and variant, except for the two violation columns, which report worst original-scale violations in the corresponding pattern–severity cell. The grid contains 20 seeds per cell. The synthetic benchmark is a conditioning and equivalence check, not a runtime benchmark.

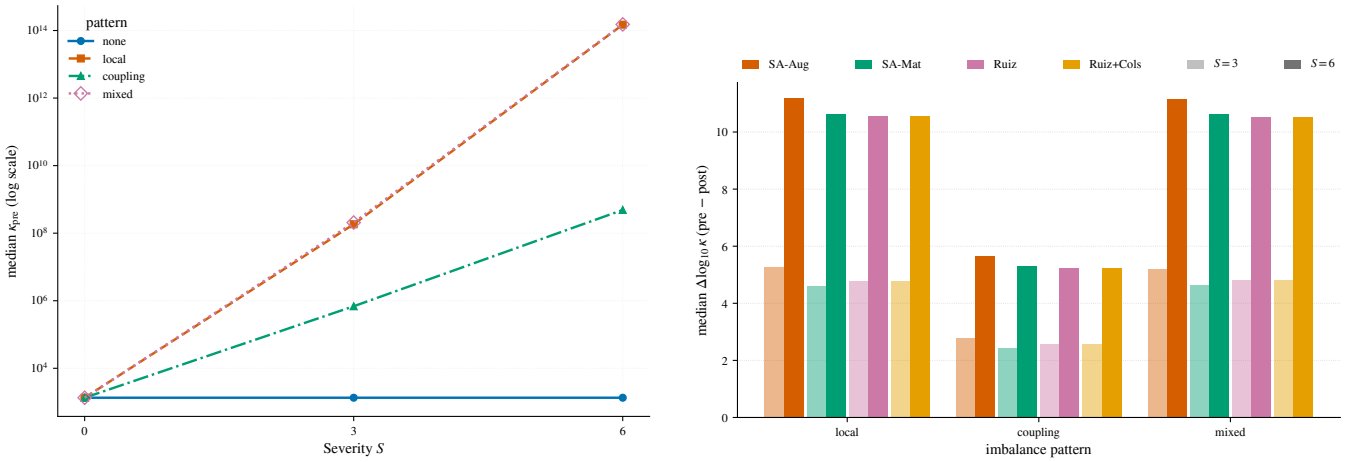
variants relative to the base is below one in Simple (0.84–0.87 at 1%, 0.80–0.81 at 0.1%) and drops by four to five orders of magnitude in Full ($\approx 5 \times 10^{-5}$ at 1%, $\approx$ 2–3 $\times 10^{-6}$ at 0.1%), while CPLEX's numerical attention score on Full falls from 0.0038 for the base to zero and the sampled unstable fraction vanishes; SG-Ter-Mer sits near parity (0.90–0.99). The numerical claim thus holds under both solvers, even though CPLEX does not convert the improved representation into runtime.

Table 14 is mainly a roadmap. It makes the solver contrast visible at a glance: structure-aware variants dominate the Gurobi rows, whereas the CPLEX rows are heterogeneous and have to be interpreted together with the paired Wilcoxon tests.

6.6 | Controlled synthetic mechanism check

The operational experiments above test the full solver–model interaction. The synthetic benchmark has a different role: it checks the numerical mechanism under controlled row-scale imbalance. The grid uses $B = 10$ blocks, each with 8 continuous and 8 binary variables, 8 local constraints plus binary-activation rows, and 3 global coupling rows with feasibility slacks. Row imbalance is injected through the pattern $\{none, local, coupling, mixed\}$ and $S \in \{0, 3, 6\}$. For each selected row r, all coefficients and the right-hand side are multiplied by $\theta_r = 10^{\xi_r}$, with $\xi_r \sim \mathcal{U}[-S, S]$. The grid contains 20 seeds per pattern–severity cell, for 240 instances and 1200 variant runs.

The design behaves as intended. For the negative-control pattern *none*, the median pre-scaling condition number is essentially invariant in S, around 1.3×10^{3}. When imbalance is injected, κ^{pre} grows with severity only in the affected row classes: at $S = 6$, it reaches about 1.5×10^{14} for *local* and *mixed*, and about 4.9×10^{8} for *coupling*. This confirms that the synthetic generator separates local-row and coupling-row imbalance by construction (figure 7a).



(a) Pre-scaling condition number by pattern and severity.

(b) Median reduction in $\log_{10}\kappa$.

FIGURE 7 Controlled synthetic mechanism check. Panel (a) verifies that the injected scale imbalance is controlled by the pattern and severity parameters. Panel (b) shows the median conditioning recovery obtained by the four scaling variants on the three imbalance patterns at $S \in \{3, 6\}$; the *none* control and the Base variant are omitted because their reduction is zero by construction (table 15).

All scaling variants recover the conditioning despite the injected severity. In the severe local and mixed cases, κ^{pre} is of order 10^{14}, whereas the post-scaling medians fall between roughly 1.1×10^3 and 4.6×10^3 (table 15). The paired median reduction in $\log_{10}\kappa$ relative to the unscaled baseline reaches about eleven orders of magnitude for *local* and *mixed* at $S = 6$, and about 5.6 orders for *coupling* at $S = 6$ (figure 7b). A paired Wilcoxon signed-rank analysis with Benjamini–Hochberg correction gives $p_{\text{BH}} \approx 1.9 \times 10^{-6}$ across the condition-number comparisons. Conversely, on *none*, scaling provides no mechanistic advantage and can slightly worsen the metric, which is the desired negative control.

The equivalence checks remain clean. Across all 240 synthetic instances, the maximum relative spread of the objective value across variants is 4.3×10^{-4}, below the solver MIPGap 10^{-3}, and no instance exceeds that threshold. In original scale, the maximum row violation is 1.3×10^{-6} and the maximum integrality violation is 2.0×10^{-6}, with no infeasible run. The synthetic benchmark therefore supports the numerical mechanism—controlled imbalance, conditioning recovery, and preservation of feasibility and objective values—while making no claim about runtime speedup.

This is the empirical bridge promised in section 3.6, and it answers the natural worry that the idealized propositions—stated for square, nonsingular blocks under exact SVD scaling—say little about a rule built from cheap coefficient proxies on rectangular, inequality-based matrices. The synthetic suite is exactly the setting where the two can be confronted, because the imbalance, and hence the conditioning, is known by construction. There the structure-keyed proxy responds to precisely the local- and coupling-row imbalance the analysis isolates: it recovers conditioning by up to eleven orders of magnitude in the local and mixed patterns, leaves the negative control essentially untouched, and never disturbs feasibility or the objective. The cheap proxy thus tracks the mechanism the idealized theory predicts on instances where that mechanism is directly observable, which is what keeps the propositions informative rather than decorative; a per-block comparison of the proxy against the true block condition number would calibrate the link quantitatively and is the natural next step.

Taken together, the four operational scenarios and the controlled synthetic check support a limited but useful interpretation. Under Gurobi, the structured preprocessing rules improve the numerical representation and often reduce solver time or deterministic effort. Under CPLEX, the same external transformations do not produce a robust runtime benefit and can be indistinguishable from, or worse than, the base in paired runtime evidence, even when some numerical diagnostics improve. The empirical message is therefore not that one scaling rule always wins. It is that generator-level structure can matter for MIP performance, but its value has to be measured through the full solver–model interaction.

TABLE 16 Solver-internal-scaling baseline (1% MIPGap; the 0.1% sweep is summarized in the text). The unscaled model is run under each solver's own scaling controls and compared, on the same instances, against the structure-aware variants under default internal scaling. Solver-time sgm (s = 1 s) and PAR10, in seconds. The "Base" row is the untransformed model under the solver's default internal scaling.

		Simple		Full		SG-Ter-Mer	
Solver	Variant	sgm (s)	PAR10 (s)	sgm (s)	PAR10 (s)	sgm (s)	PAR10 (s)
Gurobi	Base (default scaling)	3.94	11.26	109.38	3,448.2	14.09	564.3
	SA-Aug	2.13	2.50	95.24	3,349.7	8.28	554.1
	SA-Mat	2.17	2.56	102.03	3,458.1	8.64	554.6
	Internal, off	41.97	1,541.9	561.95	9,716.1	56.63	740.2
	Internal, equilibration	5.81	20.39	128.45	2,444.7	14.66	566.4
	Internal, geometric	4.40	12.60	138.11	2,497.1	20.91	589.8
	Internal, aggressive	3.79	10.79	133.83	3,469.0	21.48	580.5
CPLEX	Base (default scaling)	2.62	7.24	147.82	5,609.3	2.25	3.57
	SA-Aug	1.73	3.65	127.36	3,616.8	2.40	10.26
	SA-Mat	1.86	4.01	120.88	6,509.4	2.31	5.94
	Internal, none	2.50	3.93	(14.38)[c]	(35.5)[c]	5.17	12.29
	Internal, default (equilibration)	2.55	7.59	159.01	5,621.8	2.23	3.53
	Internal, aggressive	2.43	7.18	189.13	5,645.0	2.18	5.66

Gurobi internal modes are the `ScaleFlag` settings off/equilibration/geometric/aggressive (0/1/2/3); CPLEX modes are `Read::Scale` none/equilibration/aggressive (–1/0/1). Each solver's default (Gurobi automatic, CPLEX equilibration) is the scaling under which "Base" and the structure-aware variants run; the internal-mode rows override it on the unscaled model. The Full base sgm differs from table 6 (109 vs 83 s): with 34 instances and heavy censoring, the solved-subset geometric mean is sensitive to run-to-run variability (section 8), which is why all comparisons are within-campaign.

[c] Fast but numerically unreliable: its claimed-optimal objectives disagree with the default pipeline's beyond the stopping tolerance on 11 of 33 instances (up to 14%, once 6×), in both directions; see the text.

6.7 | Solver-internal-scaling baseline

The control of section 5.4 that most directly tests the contribution asks whether the solver's *own* scaling could reproduce the structure-aware effect. Table 16 runs the *unscaled* model under each solver's internal scaling settings and compares it, on the same instances, against the structure-aware variants run under default internal scaling.

Two facts stand out. First, scaling is not optional: disabling it (Gurobi `ScaleFlag=0`, CPLEX `Read::Scale=-1`) is catastrophic—Gurobi's sgm on Simple rises from 3.9 s to 42 s with seven timeouts, and on SG-Ter-Mer to 57 s—which confirms that the solvers' default scaling is already a strong, well-chosen baseline rather than a straw man. Second, no internal setting of either solver matches the structure-aware runtime on the Simple class: the best Gurobi flag (aggressive, 3.79 s) merely returns to the default level, and structure-aware SA-Aug (2.13 s) is 1.78× faster than it; the best CPLEX setting (aggressive, 2.43 s) also sits above SA-Aug (1.73 s), although for CPLEX this is a descriptive sgm contrast only—the paired Simple tests remain neutral to unfavorable (table 13), with median paired speedups near one. The Gurobi gain therefore does not collapse to "any extra scaling pass, and the solver could have produced it": retuning the solver's own equilibration, geometric, or aggressive modes does not recover it. This control rules out the solver's own scaler as the missing ingredient; which part of the external rule carries the effect—and in particular whether the row-role metadata itself is necessary—is the attribution question examined in section 6.8.

The SG-Ter-Mer class splits by solver. Under CPLEX the comparison is a wash—structure-aware (2.40 s) sits at the base level (2.25 s) and the aggressive internal mode (2.18 s) is marginally ahead—consistent with the rest of the study, where the Gurobi effect, not CPLEX, carries the improvement on this class (section 6.8). Under Gurobi, by contrast, the win is decisive and no internal mode comes close: structure-aware reaches 8.3 s against a 14.1 s base, the best internal setting (equilibration) only matches the base (14.7 s), and the geometric and aggressive modes are markedly *worse* than the base (20.9 and 21.5 s)—so structure-aware is 1.77× faster than the best internal mode and over 2.5× faster than the worst. The internal-scaling baseline thus supports the central claim where the external rule is robustly effective (Simple and SG-Ter-Mer under Gurobi); under CPLEX it adds a descriptive Simple contrast that the paired tests do not elevate to a robust gain, and, where the rule is not effective (SG-Ter-Mer under CPLEX), it shows structure-aware neither helping nor harming relative to the solver's tuned internal scaling.

TABLE 17 Operational stage ablation under Gurobi (1% MIPGap). The local-row stage reproduces almost the entire effect; the coupling-row stage alone is essentially inert. Solver-time shifted geometric mean (sgm, s = 1 s) and PAR10, in seconds.

Class	Variant	sgm (s)	PAR10 (s)
Simple	Base	3.73	10.47
	SA-Aug (local+coupling)	2.02	2.42
	SA-Aug, local only	2.01	2.38
	SA-Aug, coupling only	3.74	10.44
	SA-Mat (local+coupling)	2.04	2.43
	SA-Mat, local only	2.06	2.48
	SA-Mat, coupling only	3.73	10.26
Full	Base	82.55	3,379.7
	SA-Aug (local+coupling)	71.41	3,315.9
	SA-Aug, local only	70.94	3,314.4
	SA-Aug, coupling only	83.69	3,381.5
	SA-Mat (local+coupling)	78.75	4,384.9
	SA-Mat, local only	78.88	4,386.1
	SA-Mat, coupling only	90.21	3,394.1
SG-Ter-Mer	Base	14.42	565.6
	SA-Aug (local+coupling)	8.55	555.1
	SA-Aug, local only	8.58	555.2
	SA-Aug, coupling only	14.45	565.9
	SA-Mat (local+coupling)	8.96	555.6
	SA-Mat, local only	8.83	555.6
	SA-Mat, coupling only	14.43	565.8

"Local only" applies the local-row stage and leaves coupling rows at base scale; "coupling only" applies the coupling-row stage and leaves local rows at base scale. In SG-Ter-Mer, PAR10 in this campaign is dominated by a single near-limit instance that times out under all variants (unlike the main 1% scenario, where SA-Mat solves it within budget), so the attribution here is carried by sgm. In Full, three or four of the 34 instances time out per variant, so PAR10 moves in steps of one timeout and the attribution is likewise carried by sgm over the solved subset.

The Full class and the 0.1% sweep complete the picture with two honest qualifications. At 1%, Full behaves like the other classes: the structure-aware sgm (95–102 s) is below the default base (109 s) and below every internal Gurobi mode (128–562 s), although the equilibration and geometric modes save one timeout and therefore lead PAR10. At 0.1%, Simple and SG-Ter-Mer repeat the 1% conclusion (best internal settings 9.4 and 11.8 s against structure-aware 6.2–6.4 and 8.8–9.1 s), but Full does not: the retuned internal modes (337–353 s) overtake the external rule (388 s for SA-Aug, base 437 s), so on the hardest class at the strictest tolerance the structure-aware advantage holds over the default pipeline but not over a tuned internal-scaling one. The CPLEX side of the Full sweep contributes a cautionary observation rather than a baseline: disabling internal scaling (`Read::Scale=-1`) appears dramatically faster (sgm 14.4 against 147.8 s, no timeouts), but its claimed-optimal objectives disagree with the default pipeline's beyond the stopping tolerance on a third of the instances, in both directions, so we treat that configuration as numerically unreliable—a reminder, in line with the descaling audit of section 6.1, that on this class scaling choices can invalidate certificates rather than merely change runtime. The remaining CPLEX Full cells at 0.1% are partial (the four-day sweep budget expired) and are omitted.

6.8 | Operational stage attribution

The synthetic benchmark isolates local and coupling imbalance by construction; the operational ablation of section 5.4 does the same on the real hydrothermal classes by running the local-only and coupling-only variants alongside the full architecture under Gurobi. Table 17 reports the result for the three classes at the 1% tolerance (for Simple and SG-Ter-Mer, the 0.1% tolerance shows the same ordering). In the Full class several instances reach the one-hour limit under every variant, so its sgm is computed over the solved subset and PAR10 carries the censoring.

The attribution is unambiguous and consistent across the three classes (and, for Simple and SG-Ter-Mer, both tolerances): the local-only variant recovers essentially the full benefit (Simple sgm $3.73 \to 2.01$ against 2.02 for the full rule; Full $82.55 \to 70.94$ against 71.41; SG-Ter-Mer $14.42 \to 8.58$ against 8.55), whereas the coupling-only variant is statistically inert (3.74, 83.69, and

TABLE 18 Role-blind flat control under Gurobi. The same per-row geometric-mean kernel used by the local stage, applied to *every* row with no role or block metadata (single block, no coupling stage), against the full structure-aware rule and the local-only ablation. Solver-time sgm (s = 1 s), in seconds; paired speedup versus base and Cliff's δ (< 0 faster).

Class	Variant	sgm (s)	sp. vs base	Cliff's δ
Simple, 1%	SA-Aug (full)	2.02	1.00	0.00
	SA-Aug, local only	2.01	1.00	0.01
	Flat (role-blind)	2.02	0.99	0.02
SG-Ter-Mer, 1%	SA-Aug (full)	8.38	1.22	–0.34
	SA-Aug, local only	8.38	1.22	–0.34
	Flat (role-blind)	**5.25**	**1.85**	**–0.64**
SG-Ter-Mer, 0.1%	SA-Aug (full)	9.23	1.20	–0.37
	SA-Aug, local only	9.22	1.18	–0.34
	Flat (role-blind)	**5.81**	**1.86**	**–0.61**

The flat control passes the original-scale equivalence checker at the same rate as the other variants (section 6.1); its speedup is not an artifact of a different or easier optimum. At 0.1% Simple the pattern is the same as at 1% and is omitted.

14.45, all at base level). The matrix-only family behaves identically. In this testbed, therefore, the operational Gurobi effect comes from local row normalization, not from the coupling-row treatment.

Because the local stage is, in isolation, a per-row geometric mean of the coefficient extremes—a kernel computable from the flat matrix without any row-role information—the natural sharpening of this control is to apply that same kernel to *every* row, with no roles and no coupling stage. Table 18 reports this role-blind variant. In Simple it is indistinguishable from the local-only rule and from the full architecture (sgm 2.02 at 1%, matching 2.01 and 2.02; the same at 0.1%), so the row-role metadata adds nothing to runtime there. In SG-Ter-Mer it is markedly *faster* than either: sgm 5.25 against 8.38 for both the full rule and local-only at 1% (paired speedup 1.85× versus 1.22×, Cliff's δ = –0.64), and 5.81 against 9.23 at 0.1%, at the same original-scale equivalence as the other variants.

Exporting the scaled matrices identifies the mechanism precisely, and it is not the coupling treatment. In SG-Ter-Mer the model has 2160 global rows: 360 demand-balance (coupling) rows and 1800 that are global but *not* coupling—chiefly the market shortage-segment rows, whose coefficients span the segment marginal costs (here up to 4000) against a unit shortage-penalty term, with right-hand sides of order 10^5–10^7. The balance rows are already well conditioned ($\widehat{\rho} \approx 1.1$), so the coupling stage correctly leaves them unscaled, and the flat kernel leaves them unscaled too (its factor falls in the near-identity band): the coupling rows are *not* where the two rules differ, and the exported balance rows are byte-identical under both. The 1800 shortage rows, however, fall outside both stages of the structure-aware rule—they are neither local-block rows nor coupling rows—so it never scales them, leaving coefficients of order 10^3 and right-hand sides of order 10^6 in the matrix. The flat kernel, scaling every row by $1/\sqrt{M_r m_r}$ over its coefficients *and* right-hand side, brings each of them to order one; on this instance that lowers the exported condition-range proxy from 3.7×10^{11} to 1.3×10^{10}, and it is the only difference between the two scaled matrices. The runtime lever under Gurobi is therefore *complete* per-row normalization: the structure-aware decomposition into local and coupling roles left a coverage gap over exactly the highest-magnitude rows, and the role-blind kernel wins by closing it, without using any metadata. The structure-aware coupling treatment, which drives the conditioning diagnostics (table 8), is not what drives the runtime. This is the cleanest form of the dissociation developed in section 7: the rule that best improves solver-facing conditioning is not the rule that solves fastest. The demonstrated value of the generator metadata is in the numerical representation; the practical accelerator is a role-blind, full-coverage normalization the metadata is not needed to build.

6.9 | Hyperparameter sensitivity

Table 19 varies the two constants of the structure-aware rule—the near-identity band u (equation (29)) and the admissible coefficient window (equation (30))—on the Simple class under Gurobi, holding the other at its default.

The runtime behavior of the structure-aware rule is insensitive to both constants: the solver-time sgm stays within $[2.01, 2.12]$ s across the entire range, a span an order of magnitude smaller than the improvement over the base (3.73 s). The conclusions of the study therefore do not depend on the specific near-identity band or coefficient window chosen, and the modest differences between settings are within run-to-run noise.

TABLE 19 Hyperparameter sensitivity of SA-Aug (Simple, Gurobi, 1% MIPGap). Solver-time sgm (s = 1 s) and PAR10, in seconds; default settings in bold.

Parameter	Value	sgm (s)	PAR10 (s)
Band u	1.5	2.09	2.46
	2	2.07	2.45
	4	2.12	2.49
Window $[\varepsilon_{\min}, \varepsilon_{\max}]$	$[10^{-6}, 10^{6}]$	2.04	2.40
	$\mathbf{[10^{-9}, 10^{9}]}$	2.02	2.39
	$[10^{-12}, 10^{12}]$	2.01	2.36

Base sgm is 3.73 s for reference. Each row is a separate matched run; the spread across all settings (sgm $\in$ [2.01, 2.12]) is far below the base-to-scaled gap, and the median paired speedup is $\approx$ 1.0 throughout. The number of Ruiz iterations K is a parameter of the Ruiz baselines, not of the structure-aware rule, so it is fixed at $K = 4$; varying it leaves SA-Aug unchanged, as expected.

7 | DISCUSSION

The experiments support a bounded claim. Structure-aware positive row scaling changes the solver-facing representation in the intended direction: under Gurobi this often reduces robust runtime or deterministic effort, whereas under CPLEX 22.1.2.0 the same external scaling yields no robust runtime gain. The evidence thus separates three issues that are easily conflated: algebraic equivalence, numerical representation, and mixed-integer search performance.

Table 20 keeps the claim bounded: not universal acceleration, but a reproducible way to use block and coupling information before export and a test of whether each solver pipeline benefits from it.

A first objection is that commercial solvers already scale internally—which is exactly why internal scaling and presolve stay enabled in all experiments. The comparison is not external scaling versus a naive solver, but whether a generator can improve the representation before the solver's own pipeline begins. The mixed Gurobi–CPLEX outcome is informative: the same representation can help one pipeline and be absorbed by another.

A second objection is that any benefit may come from generic coefficient compression rather than from generator information. The Ruiz baselines are most informative precisely here, and they are not a weak foil: max-norm Ruiz is a strong, standard equilibrator that does compress coefficient ranges—yet its matched runtime is irregular and does not reliably turn that compression into speed. Coefficient compression on its own is therefore not the lever. The decisive control is sharper still: the comparison against the solver's *own* internal scaling (sections 5.4 and 6.7), since on the classes where the external rule is effective no tuned internal setting reproduces the structure-aware runtime on the same instances (on Full at the strictest tolerance the tuned internal modes do overtake it; section 6.7). The objection is addressed by that control together with the Ruiz contrast—a generic compressor and the solver's own scaler both fall short of the structure-keyed rule where that rule delivers its gains.

A mechanistic caveat must accompany the conditioning evidence. If better conditioning drove faster search, the best-conditioned variant would be fastest, but the data do not line up that way: the augmented rule improves solver-level conditioning more than the matrix-only rule, yet the matrix-only rule wins runtime more often in Simple and SG-Ter-Mer, and the association between the conditioning ratio and speedup is weak (a small Spearman correlation explaining a few percent of variance). The honest reading is that conditioning is at best a partial mediator: the representation change is real and in the intended direction, but the runtime benefit is plausibly routed through mechanisms the metric does not capture—which presolve reductions fire, or how many simplex bound-flips and node relaxations the scaled model induces. This reading has precedent: large-scale LP experiments find the runtime effect of scaling irregular even where conditioning improves [15], and the conditioning of the LP relaxations is known to evolve along the branch-and-cut tree in ways a static snapshot of the exported matrix cannot summarize [20]. We therefore present conditioning as evidence that the representation improved, not as the proven cause of the speedup. The role-blind flat control (table 18) makes the dissociation sharpest: applying the plain per-row kernel to every row, with no metadata, matches the structure-aware rule in Simple and beats it in SG-Ter-Mer, so the rule that most improves conditioning is not the one that solves fastest, and the runtime accelerator is a cheap normalization that does not need the row roles. The generator metadata earns its place in the numerical representation—where the structure-aware coupling treatment measurably lowers the solver-level conditioning ratios—rather than as the source of the runtime gain.

TABLE 20 Claim, evidence, and limitation are distinct parts of the argument.

Claim	Evidence in this study	Limitation
Algebraic equivalence	Positive factors multiply complete constraints; post-solve checks evaluate feasibility, integrality, and objectives in the original scale.	This claim concerns the mathematical MIP, not the path followed by branch-and-bound.
Numerical representation	Structure-aware variants reduce solver-facing conditioning diagnostics in the harder classes.	Conditioning diagnostics describe representation; they do not prove faster MIP search.
Gurobi performance	Per-row local scaling improves robust runtime or deterministic-effort metrics in the main Gurobi scenarios.	The lever is the per-row kernel, not the row-role metadata: a role-blind flat control matches or beats the structure-aware rule on runtime (table 18); the claim is conditional on the tested pipeline.
Solver dependence	CPLEX remains near parity or deteriorates in paired runtime evidence, even when some numerical diagnostics improve.	The result is tied to CPLEX 22.1.2.0, default settings, and the ClusterUY environment.
Generalization	Generator-level block information can guide numerical preprocessing before export; the synthetic benchmark injects local and coupling imbalance by construction and confirms the intended conditioning response.	Runtime evidence still comes from one hydrothermal dispatch platform; the synthetic benchmark is a mechanism check, not a performance benchmark.

The table keeps the main characters of the argument visible: the algebraic transformation, the numerical representation, the solver pipeline, and the scope of the evidence.

A third objection is that the right-hand side may be unnecessary, and the evidence supports it. SA-Aug uses coefficients and right-hand sides and stays strongest in the Full class, yet SA-Mat recovers much of the benefit from the coefficient matrix alone and is often best in Simple and SG-Ter-Mer. In this testbed the local–coupling distinction matters more than including the right-hand side in every scale proxy; other generated models may differ, but the present gains do not require it.

A fourth objection comes from CPLEX: several transformations improve static or solver-level diagnostics, yet paired runtime evidence stays neutral to unfavorable. This does not weaken the numerical claim; it limits the performance claim. A commercial MIP run is not a linear-algebra experiment—presolve, internal scaling, relaxation bases, cuts, incumbents, branching, heuristics, and tolerances all stand between the exported matrix and the runtime—so external scaling may improve one stage while the binding bottleneck lies elsewhere. We probed the most natural such candidate—that CPLEX's presolve neutralizes the external scaling—by re-running the Simple class under CPLEX with presolve disabled. The per-instance effect stayed essentially neutral in both regimes (median paired speedup ≈ 1.00, with the structure-aware variant faster on about half of the 204 instances), so the absence of a robust CPLEX gain is not a presolve artifact: it persists with presolve on or off. We therefore report the CPLEX neutrality as an intrinsic, scoped finding rather than an effect that a different internal configuration would recover.

The methodological lesson: numerical preprocessing for MIPs should be judged on four separable layers—algebraic equivalence, numerical diagnostics, matched runtime, and solver contrast—and never on one coefficient-range statistic or one runtime table. The study reports all four layers and keeps their claims separate.

The controlled synthetic benchmark closes part of the gap: when the source of imbalance is known by construction, the variants respond as intended and preserve original-scale feasibility and objective values. It does not close the performance gap—the synthetic instances solve in milliseconds—so runtime claims still come from operational instances, and the operational stage ablation (section 5.4) separates local-row normalization from the coupling-row treatment on real data. Other domains (logistics, production planning, network design) would further test whether the mechanism persists as the generator structure changes; it should transfer only insofar as it shares the premise of sparse global coupling over local blocks.

7.1 | Deployability and downstream effects

The method presupposes that the metadata it needs—row role, block ownership, and coupling incidence—are available at model-generation time. This is immediate in a generator that the team owns, as here, but it is not free in the mainstream algebraic modeling languages (Pyomo, JuMP, GAMS, AMPL) that many practitioners use, which typically export a flat matrix and do not surface component structure. Adoption therefore divides into two cases. A team that owns its generator can emit the three metadata fields at low cost, since the generator already iterates over components when it builds the rows. A team that does not own its generator would need either to instrument the export path or to *recover* block structure after export, for example by hypergraph or matrix partitioning of the kind used by automatic Dantzig–Wolfe structure detectors[21]; such a fallback widens applicability beyond owned generators at the price of an approximate, detected block map. The practical value proposition is correspondingly narrow and concrete: for a Gurobi user running many hard instances of a generated model whose

structure is known, the method can reduce timeouts at near-zero marginal cost, with no benefit demonstrated for CPLEX and no per-instance speedup promised on easy instances.

A second deployment caveat concerns quantities other than the optimal point. Positive row scaling multiplies each constraint, and therefore its dual, by the inverse of its row factor: the constraint dual reported by the solver for a scaled row is the original shadow price divided by that factor. Operations workflows that read shadow prices—marginal costs of demand balances, water values, reserve prices—must descale the duals by the recorded row factors to recover original-scale sensitivity information, exactly as the primal solution is descaled before the feasibility checks. Because the row factors are stored for the audit trail, this descaling is mechanical; but it must be applied, or downstream dual interpretation will be off by the per-row scale.

8 | STRENGTHS, LIMITATIONS, AND REPRODUCIBILITY

The study is strongest where the evidence is matched: runtime comparisons use the same instance, solver, and tolerance for base and variants; numerical diagnostics are reported separately from speed; and feasibility and integrality are checked in the original scale. These choices keep the argument auditable—a reader can see which evidence supports algebraic equivalence, which supports numerical improvement, and which supports solver-dependent acceleration.

The main limitation is domain coverage. The method applies to generated modular MIPs with identifiable local blocks and coupling rows, but the evidence comes from one hydrothermal dispatch platform. That platform supplies a realistic and numerically heterogeneous testbed. It does not prove that the same behavior will appear in other generated optimization domains.

A second limitation is component-level attribution within the operational testbed. The core comparison evaluates the full local–block–coupling architecture against right-hand-side and generic-scaling alternatives; the operational stage ablation introduced in section 5.4 adds the local-only and coupling-only variants on the real hydrothermal classes specifically to localize the effect to a stage. To the extent that the ablation cannot cleanly separate the stages—because the stages interact, the coupling-row factor being defined relative to the already-scaled blocks—the residual limitation is one of attribution, not of validity: every variant is still an exact reformulation of the same MIP, so the solver-dependent performance pattern stands regardless of how the credit splits between stages.

A third limitation is the choice of the row as the unit of intervention. The method scales complete rows, which keeps the transformation trivially equivalence-preserving and leaves variable bounds and integrality untouched. It does not, however, test the competing hypothesis that *column*-scale heterogeneity—water volumes, flows, costs, and power mixed across variables—is the larger numerical lever in these models. Generic column scaling is included only inside the Ruiz+Cols baseline, and the synthetic generator injects row imbalance by construction, so it cannot, by design, reveal whether a structured *column*-scaling counterpart would help more. The row-only framing is thus a deliberate scoping decision, not an established claim that row imbalance dominates; a structure-aware column-scaling rule is a natural and untested companion to the present work.

Solver dependence is another threat. With internal presolve and scaling left enabled, the measured effect is that of an external layer inside realistic commercial pipelines—relevant to practice, but conditional on solver version, settings, and environment. The negative CPLEX evidence is therefore a finding about the tested pipeline, not a universal statement about CPLEX.

A related threat is run-to-run performance variability. Each instance–variant–scenario cell is measured by a single run under default solver seeds, so the well-documented variability of MIP performance is not averaged out. The repeated base campaigns bound it empirically: across the main scenario, the internal-scaling baseline, and the stage ablation (tables 6, 16 and 17), the Simple base sgm spans 3.73–4.05 s and the SG-Ter-Mer base 14.09–14.42 s—a run-to-run drift of under 10%, an order of magnitude smaller than the 1.8–2$\times$ structure-aware effects, and the deterministic-effort metrics (`Work` units, ticks) are insensitive to it. The Full class drifts more (base sgm 82.5–109.4 s across the same campaigns): with 34 instances and heavy censoring, its solved-subset geometric mean is fragile, which is why every Full conclusion in this paper rests on within-campaign matched pairs rather than on cross-campaign level comparisons. Seed-replicated runs would nonetheless be a stronger control and are left as future work.

The numerical diagnostics have limited reach: exported coefficient ranges are static matrix summaries, and solver-level conditioning depends on the state the solver exposes and on auxiliary fixed-integer LP computations; neither measures full MIP difficulty. The paper therefore reports original-scale feasibility, integrality after descaling, objective checks, performance profiles, deterministic effort, paired tests, and primal–dual progress as complementary evidence.

Finally, the experiments use operational data whose raw inputs may not be fully redistributable, so the reproducibility package includes public code, aggregate results, paired-instance identifiers, processed logs, and rebuild scripts. The controlled

synthetic benchmark (sections 5.2 and 6.6) is an auditable complement when raw data cannot be released: it reproduces the numerical mechanism without claiming operational speedup.

9 | CONCLUSION

Generated MIPs contain useful structure before export—local rows, component blocks, and coupling rows—that the solver usually never sees beyond the flat matrix. This paper uses that pre-export information to choose positive row scalings that preserve the optimization problem but change its numerical representation.

The answer to the research question is qualified but affirmative: structure-aware positive row scaling improves solver-facing numerical diagnostics for the tested block-structured hydrothermal MIPs, and under Gurobi this often coincides with lower robust runtime or deterministic effort, whereas under CPLEX 22.1.2.0 it does not. The method improves representation; acceleration remains solver-dependent.

The contribution divides along the same line the evidence does. The local–block–coupling architecture—scale local rows first, summarize blocks second, scale coupling rows last with the scales of the blocks they connect—uses information the generator has but a flat-matrix routine may not, and, because each factor multiplies a complete constraint by a strictly positive scalar, feasible points, objective values, and integer restrictions are unchanged. On the *numerical* side that architecture earns its keep: its coupling treatment lowers the solver-facing conditioning diagnostics more than any baseline. On the *runtime* side the attribution is more sober. The stage ablation locates the Gurobi speedup in the local stage, and the role-blind flat control (table 18) shows that the same per-row kernel applied to every row, with no metadata, matches the structure-aware rule in Simple and is substantially faster in SG-Ter-Mer—because the role-based rule left a coverage gap over the high-magnitude global rows that belong to no local block and are not coupling rows, exactly the rows the flat kernel reaches. The practical accelerator is therefore a cheap, complete per-row normalization that does not require the row roles, and the structure metadata, valuable for the representation, is not the source of the runtime gain. Establishing even this bounded result required separating four kinds of evidence—algebraic equivalence, original-scale feasibility, solver-facing diagnostics, and matched per-solver runtime—which is as much a methodological takeaway as the scaling rule itself: the rule that best conditions the matrix is not the rule that solves fastest, and only matched per-solver runtime tells them apart.

Future work should test the mechanism beyond hydrothermal dispatch—larger synthetic suites and applications in logistics, production planning, and network design would show whether the pattern persists as the engineering structure changes. A concrete lesson for structure-driven preprocessing also follows from the coverage gap identified here: a rule that scales only the rows it recognizes by role can silently miss the worst-scaled rows, so completeness of coverage, not role sophistication, is what a practical scaling layer should guarantee.

AUTHOR CONTRIBUTIONS

Mathias Rodríguez Castro: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review and editing.

ACKNOWLEDGMENTS

The author acknowledges Universidad de la República and the computational resources used for the experimental evaluation. The experiments presented in this paper were carried out using ClusterUY (https://cluster.uy)[17]. The author thanks Claudio Risso and Ignacio Ramírez for supervision, methodological discussions, and feedback during the development of this work. The MIP formulation used in this study belongs to a broader line of work on hydroelectric and hydrothermal dispatch models developed by Claudio Risso and collaborators.

FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

CONFLICTS OF INTEREST

The author declares no conflicts of interest.

ETHICS STATEMENT

This study consists of computational experiments and does not involve human participants, animal subjects, patient data, or clinical-trial data.

DATA AVAILABILITY STATEMENT

The reproducibility package for this study is publicly available at https://github.com/MathiasRodriguezCastro/structure-aware-row-scaling and archived on Zenodo at https://doi.org/10.5281/zenodo.20648950. The repository contains the cleaned experimental source code, preprocessing scripts, the redistributable dispatch input instances used in the paper, the synthetic benchmark instances, aggregate result tables, paired-instance identifiers, and the scripts used to rebuild the reported tables and figures. Large per-instance solver logs and bulky dispatch CSVs are not kept in the Git tree only to keep the repository lightweight; they are distributed as external archives linked from the repository's `artifacts/README.md`. The repository also documents the solver versions, scenario definitions, command-line options, time limits, MIPGap settings, preprocessing flags, and log-parsing rules used to construct the matched comparisons. The aggregate results were produced by the original ClusterUY runs, while the released C++ code is a cleaned reproducibility version containing the monolithic MIP path and preprocessing methods used in the paper.

SUPPORTING INFORMATION

A Supporting Information document accompanies this article with the 0.1%-MIPGap Gurobi distributional, diagnostic, and performance-profile views (Figures S1–S3). Additional processed solver logs, extended tables, diagnostic plots, and scripts for rebuilding the numerical evidence are available in the reproducibility repository or as external artifacts linked from it.

References

1. Risso C, Nesmachnow S, Porteiro R, Vignolo JM, Sierra E, Ibarburu M. A Mixed Combinatorial Optimization Model for the Río Negro Hydroelectric Complex. In: Springer Proceedings in Energy. Springer. 2025; Singapore:403–420. Presented at ICSC-Cities 2024, San Carlos, Costa Rica

2. Risso C, Nesmachnow S, Porteiro R, Vignolo JM, Sierra E, Ibarburu M. A Mixed Combinatorial Optimization Model to Manage the Hydrothermal Dispatch for the Uruguay River Hydroelectric Plant. In: ICSC-Cities. 2025; Puebla, México. Preproceedings paper; the Springer Proceedings in Energy volume is in press. Available at https://www.colibri.udelar.edu.uy/jspui/handle/20.500.12008/53524, accessed July 4, 2026.

3. Rodríguez Castro M, Montaño Cobas ME, Zampetti Garin E. Implementación de Modelo de Despacho Eléctrico para el Complejo Hidroeléctrico del Río Negro. tesis de grado. Universidad de la República, Facultad de Ingeniería, Instituto de Computación. Montevideo, Uruguay: 2025.

4. Wood AJ, Wollenberg BF, Sheblé GB. *Power Generation, Operation, and Control*. Wiley. 3 ed., 2013.

5. Conejo AJ, Carrión M, Morales JM. *Decision Making Under Uncertainty in Electricity Markets*. Springer, 2010

6. Vielma JP. Mixed Integer Linear Programming Formulation Techniques. *SIAM Review.* 2015;57(1):3–57. doi: 10.1137/130915303

7. Risso C, Nesmachnow S, Vignolo M, et al. An Enriched Mixed Combinatorial Optimization Model to Manage the Hydrothermal Dispatch for the Río Negro Hydroelectric Complex. *Programming and Computer Software.* 2026;52(2):180–201. doi: 10.1134/S0361768826700209

8. Klotz E, Newman AM. Practical Guidelines for Solving Difficult Linear Programs. *Surveys in Operations Research and Management Science.* 2013;18(1–2):1–17. doi: 10.1016/j.sorms.2012.11.001

9. Klotz E, Newman AM. Practical Guidelines for Solving Difficult Mixed Integer Linear Programs. *Surveys in Operations Research and Management Science.* 2013;18(1–2):18–32. doi: 10.1016/j.sorms.2012.11.002

10. Tomlin JA. On Scaling Linear Programming Problems. *Mathematical Programming Studies.* 1975;4:146–166. doi: 10.1007/BFb0120718

11. Curtis AR, Reid JK. On the Automatic Scaling of Matrices for Gaussian Elimination. *IMA Journal of Applied Mathematics.* 1972;10(1):118–124. doi: 10.1093/imamat/10.1.118

12. Sinkhorn R, Knopp P. Concerning Nonnegative Matrices and Doubly Stochastic Matrices. *Pacific Journal of Mathematics.* 1967;21(2):343–348. doi: 10.2140/pjm.1967.21.343

13. Ruiz D. A Scaling Algorithm to Equilibrate Both Rows and Columns Norms in Matrices. Tech. Rep. RAL-TR-2001-034, Rutherford Appleton Laboratory; Oxfordshire, United Kingdom: 2001.

14. Achterberg T, Bixby RE, Gu Z, Rothberg E, Weninger D. Presolve Reductions in Mixed Integer Programming. *INFORMS Journal on Computing.* 2020;32(2):473–506. doi: 10.1287/ijoc.2018.0857

15. Elble JM, Sahinidis NV. Scaling Linear Optimization Problems Prior to Application of the Simplex Method. *Computational Optimization and Applications.* 2012;52(2):345–371. doi: 10.1007/s10589-011-9420-4

16. Dolan ED, Moré JJ. Benchmarking Optimization Software with Performance Profiles. *Mathematical Programming.* 2002;91(2):201–213. doi: 10.1007/s101070100263

17. Nesmachnow S, Iturriaga S. Cluster-UY: Collaborative Scientific High Performance Computing in Uruguay. In: Torres M, Klapp J. , eds. *Supercomputing*, , . 1151 of *Communications in Computer and Information Science*. Cham: Springer, 2019:188–202

18. Gurobi Optimization, LLC . Gurobi Optimizer Reference Manual, Version 13.0.1. https://www.gurobi.com/documentation/; 2026. Accessed July 4, 2026.

19. IBM Corporation . IBM ILOG CPLEX Optimization Studio Documentation, Version 22.1.2.0. https://www.ibm.com/docs/en/icos; 2026. Accessed July 4, 2026.

20. Miltenberger M, Ralphs T, Steffy DE. Exploring the Numerics of Branch-and-Cut for Mixed Integer Linear Optimization. In: Springer. 2018; Cham:151–157

21. Bergner M, Caprara A, Ceselli A, et al. Automatic Dantzig–Wolfe Reformulation of Mixed Integer Programs. *Mathematical Programming.* 2015;149(1–2):391–424. doi: 10.1007/s10107-014-0761-5